

SE-801: Software Project Lab III

Final Project Report

CodePulse

Intelligent Multi-Platform Coding Performance Analyzer

Software Design · Implementation · Testing · User Manual

Submitted by

Abdus Salam Islam Badhon

Student ID: BSSE-1401

Supervised by

Mohd. Zulfiqar Hafiz

Professor, Institute of Information Technology
University of Dhaka

Signature of the supervisor:

Source repository: <https://github.com/Badhon1401/SPL-3>

October 2026

Abstract

Competitive programmers and computer-science students practise across several online judges, yet each platform shows only its own isolated statistics. Learners therefore lack a combined performance view, cross-platform weakness detection, personalised practice guidance and any visibility of behavioural trends such as streak loss or comfort-zone stagnation.

CodePulse addresses these gaps as a three-tier web application. A Next.js 14 frontend presents dashboards and charts; a Spring Boot 3.3 backend synchronises submissions from Codeforces, LeetCode, AtCoder and CodeChef, stores them in PostgreSQL, and computes a unified **CodePulse Rating** with a weighted Elo-like algorithm, combined streaks, topic-level weakness scores and automatically generated hidden insights. An algorithmic recommendation engine selects unsolved problems from a user's weakest topics, and an **AI Coach** built on Mistral AI converts free-text goals into personalised problem lists grounded in the user's measured history. Security is provided by BCrypt and stateless JWT authentication; data growth is bounded by a 300-submission pruning policy; and external failures are isolated so that one unavailable platform never stops the others.

This report documents the complete project: requirements, system and data modeling, component-level design (design classes, persistent data sources, behavioural representations, deployment and alternatives), interface design, implementation, AI engineering, a 25-case test suite, a user manual, installation guide and API reference. The source code is available at <https://github.com/Badhon1401/SPL-3>.

Keywords: competitive programming, performance analytics, recommendation engine, large language model, Spring Boot, Next.js, JWT, PostgreSQL.

Table of Contents

Note: if page numbers are not shown, right-click the table and choose "Update Field" (or press F9).

1. Introduction

1.1 Project Overview

CodePulse is a full-stack web application that acts as an analytical intelligence layer above the major competitive-programming platforms — Codeforces, LeetCode, AtCoder and CodeChef. Instead of forcing a learner to read four separate statistics pages, CodePulse synchronises their submission history, normalises it into a single relational model, and derives insights that none of the individual platforms provide: a unified **CodePulse Rating**, cross-platform topic-level weakness scores, a combined streak, automatically generated "hidden insights", algorithmic problem recommendations, and an **AI Coach** that turns a free-text goal into a personalised practice plan.

The system is built as a three-tier application. The presentation tier is a Next.js 14 single-page style interface; the application tier is a Spring Boot 3.3 REST backend that contains the synchronisation, analytics, recommendation and AI-integration engines; and the data tier is a PostgreSQL database. Stateless JWT authentication secures every layer boundary.

This document is the **final report** for the SE-801 project course. It consolidates the earlier deliverables (proposal and midterm SRS) and extends them with the component-level design, interface design, implementation details, testing documentation, user manual and installation guide required for the final defence.

1.2 Problem Statement

Competitive programmers and software engineering students practise on several platforms at once. Each platform operates in isolation and presents only its own statistics. As a result learners face the following concrete problems:

- There is **no single place** to see combined solved-problem counts, acceptance rates or activity trends across all platforms.
- There is **no automated weakness detection** that spans platforms — for example, repeatedly failing Dynamic Programming problems on both Codeforces and LeetCode.
- Choosing the next problem is **ad hoc**. Users cannot state a goal in natural language and receive a personalised, history-aware practice list.
- There is **no unified rating** that reflects competence across all connected platforms.
- Behavioural patterns — declining activity, comfort-zone stagnation, streak loss — are **invisible** without manual record keeping.

1.3 Motivation

Mainstream platforms excel at hosting problems and contests but give little structured feedback on *how* a user is improving. Learning becomes inefficient and repetitive when practice is not guided by data. CodePulse targets three groups: competitive programmers who want to level up, computer-science students preparing for assessments, and job candidates preparing for technical interviews. The core purpose is to convert scattered statistics into intelligent, actionable guidance for structured skill development.

1.4 Objectives

- Build a full-stack web application that aggregates data from Codeforces, LeetCode, AtCoder and CodeChef into one dashboard.
- Compute a unified **CodePulse Rating** with a weighted Elo-like algorithm.

- Implement automated topic-level weakness detection and a curated problem recommendation engine with a feedback loop.
- Integrate a Large Language Model (Mistral AI) to accept free-text coaching prompts and return recommendations grounded in the user's real performance history.
- Generate **hidden analytical insights** — patterns the user would not otherwise notice.
- Provide scheduled (6-hourly) and on-demand synchronisation with a configurable ceiling of 300 stored submissions per user.
- Secure the system with BCrypt password hashing, stateless JWT authentication and role-based authorisation.

1.5 Scope

Table 1: Project scope

In scope	Out of scope
Registration, login, profile with four platform handles Synchronisation with Codeforces REST, LeetCode GraphQL, AtCoder (kenkoooo) and CodeChef community APIs Combined analytics, rating, streaks, heatmap, per-platform breakdown, hidden insights Algorithmic recommendations AI Coach with persisted sessions Automatic 6-hour sync, submission pruning	Contest scheduling and reminders Real-time editorial fetching Social / collaborative features Native mobile application Executing or judging user code

1.6 Deliverables

- Full-stack application: Spring Boot 3.3 backend and Next.js 14 frontend.
- PostgreSQL schema with 7 core tables (plus the `problem_topics` join table).
- REST API with 17 documented endpoints.
- This final report (design, implementation, testing, user manual, repository URL).
- Source repository: <https://github.com/Badhon1401/SPL-3>
- Installer: not applicable — the system is run from source / packaged JAR (see Chapter 12).

1.7 Organisation of this Report

Table 2: Report organisation

Chapter	Content
2	Background and related work
3	Requirements analysis (functional, non-functional, stakeholders, constraints)
4	System modeling — use cases and activity diagrams
5	Data and information modeling — schema and ER diagram
6	Component-level design — design classes, persistent data, behaviour, deployment, algorithms, alternatives and refactoring
7	Interface design — objects, actions, events, states and screens
8	Implementation details
9	AI engineering design
10	Testing — strategy, criteria, risks, test cases and results
11	User manual
12	Installation and deployment guide

Chapter	Content
13	API reference
14	Project management, challenges and lessons learned
15	Conclusion and future work; references; appendices

2. Background and Related Work

2.1 The Competitive Programming Ecosystem

The four platforms integrated by CodePulse differ in how they expose data, which strongly shaped the synchronisation design:

Table 3: *Supported platforms and their data sources*

Platform	Data source used by CodePulse	Characteristics
Codeforces	Public REST API — <code>user.status</code> , <code>user.rating</code>	Rich, structured verdicts, problem tags and difficulty rating; contest-based rating history.
LeetCode	Public GraphQL endpoint — recent submission list	Difficulty is a label (Easy / Medium / Hard) rather than a numeric rating; interview-oriented tags.
AtCoder	Community API (kenkoooo.com) — submissions	Numeric problem difficulty estimates; contest-driven; third-party data source.
CodeChef	Community wrapper API	Difficulty bands; community-maintained API, therefore less stable.

2.2 Limitations of Existing Tools

Each platform provides a profile page with solved counts and, for some, a rating graph. These views are platform-local: they cannot combine activity across sites, they do not rank a user's weak topics against each other, and they do not explain behavioural trends. Third-party dashboards exist for individual platforms, but a unified, recommendation-oriented analyser that also accepts natural-language goals is not available as a simple self-hosted service. CodePulse fills this gap.

2.3 Positioning of CodePulse

Table 4: *Capability comparison*

Capability	Single-platform profile	CodePulse
Cross-platform aggregation	No	Yes — four platforms, one dashboard
Unified rating	No	Yes — weighted Elo-like CodePulse Rating
Topic weakness scoring	Limited / manual	Automatic failure-rate score per topic
Hidden insights (comfort zone, streak drop, imbalance)	No	Yes — 6–10 categorised findings
Personalised recommendations	Generic problem lists	Algorithmic + AI Coach grounded in history
Natural-language practice goals	No	Yes — Mistral AI

2.4 Technologies Reviewed

Spring Boot was chosen for the backend because it provides production-grade security, data-access and scheduling support with minimal boilerplate. **PostgreSQL** gives strong relational guarantees for the many-to-many Problem–Topic model and efficient aggregate queries. **Next.js with React** provides routing,

server/client component separation and a mature charting ecosystem (Recharts). For AI integration, direct REST calls to the Mistral chat-completion endpoint were preferred over an SDK after the Spring AI milestone releases failed to tolerate additive response fields (see Section 6.9).

3. Requirements Analysis

3.1 Functional Requirements

3.1.1 Authentication and User Management

Table 5: *Authentication and user management requirements*

ID	Requirement	Use case
FR-1	Users can register with username, email and password; passwords are stored BCrypt-hashed.	UC-1
FR-2	Users can log in and receive a JWT access token with a 24-hour expiry.	UC-1
FR-3	Users can store and update Codeforces, LeetCode, AtCoder and CodeChef handles in their profile.	UC-1
FR-4	Platform handles are always fetched fresh from the database on profile and dashboard load.	UC-1

3.1.2 Data Synchronisation

Table 6: *Data synchronisation requirements*

ID	Requirement	Use case
FR-5	Fetch up to 200 recent submissions per platform per sync from Codeforces (<code>/user.status</code>), LeetCode (GraphQL), AtCoder (kenkoooo) and CodeChef (community API).	UC-2
FR-6	Deduplicate submissions by <code>platformSubmissionId</code> before storage.	UC-2
FR-7	If a user's stored submissions exceed 300, prune the oldest so only the most recent 300 remain.	UC-2
FR-8	Synchronise automatically every 6 hours via a scheduled job and allow manual triggering.	UC-2

3.1.3 Analytics

Table 7: *Analytics requirements*

ID	Requirement	Use case
FR-9	Compute a combined CodePulse Rating from all connected platforms using a weighted-average algorithm.	UC-3
FR-10	Compute current streak, longest streak and total active days across all platforms combined.	UC-3
FR-11	Compute topic-level weakness scores (failure rate per topic) from all submissions.	UC-3
FR-12	Generate 6–10 hidden insights automatically (weakness, comfort zone, streak pattern, activity trend, platform imbalance ...).	UC-3
FR-13	Return per-platform statistics (solved count, acceptance rate, verdict distribution, topic coverage) alongside combined statistics.	UC-3

3.1.4 Recommendations

Table 8: *Recommendation requirements*

ID	Requirement	Use case
FR-14	Select up to 10 unsolved problems from the user's weakest topics within the estimated difficulty range.	UC-4
FR-15	Generating new recommendations replaces all previous ones for that user.	UC-4

ID	Requirement	Use case
FR-16	Users can mark a recommendation as solved (feedback loop) or dismiss it.	UC-4

3.1.5 AI Coach

Table 9: AI Coach requirements

ID	Requirement	Use case
FR-17	Users can submit a free-text prompt describing a practice goal.	UC-5
FR-18	Build a context payload (rating, weakness scores, per-platform stats, recently solved problems, recent WA/TLE topics) and send it with the prompt to Mistral AI.	UC-5
FR-19	Mistral returns structured JSON: problem recommendations, a coach insight paragraph and focus areas.	UC-5
FR-20	Persist AI sessions; the latest session is shown on page load without regeneration.	UC-5
FR-21	Regeneration deactivates the previous session and creates a new one.	UC-5
FR-22	Users can mark AI-recommended items as solved or dismiss them.	UC-5

3.2 Non-Functional Requirements

Table 10: Non-functional requirements

ID	Attribute	Requirement	Measure
NFR-1	Performance	Analytics responses for a user with up to 300 submissions are returned quickly.	< 2 s
NFR-2	Security	All endpoints except <code>/api/auth/**</code> require a valid JWT; passwords never stored in plaintext.	401 without / with invalid token
NFR-3	Reliability	A failure on one platform must not crash the overall sync job.	Error caught and logged; other platforms continue
NFR-4	Scalability	Database growth per user is bounded; syncs are asynchronous.	≤ 300 submission rows per user
NFR-5	Usability	Loading skeletons, toast notifications and empty states with action prompts.	Manual acceptance test
NFR-6	Maintainability	Each platform sync is an independent service class.	New platform = 1 service + 1 handle field
NFR-7	AI correctness	LLM prompt yields strictly valid JSON; responses parsed through <code>JsonNode</code> .	Tolerates additive schema changes
NFR-8	Portability	Runs on any OS with Java 17, PostgreSQL 15 and Node.js 18.	Verified on development machine

3.3 Stakeholders

Table 11: Stakeholders

Stakeholder	Role	Interest in CodePulse
Competitive programmer	Primary user	Improve problem-solving skills efficiently
CS student	Primary user	Prepare for coding interviews, contests and assessments
Supervisor / evaluator	Academic oversight	Assess technical depth, design quality and engineering rigour
Developer / maintainer	Builder	Clean architecture and an extensible platform
External platform providers	Data source	Fair-use, rate-limited API access

3.4 Assumptions and Constraints

- Platform handles are supplied by the user; CodePulse reads only **public** submission data and never needs platform passwords.
- Public APIs may rate-limit or change; the design therefore isolates each platform behind its own service and tolerates partial failure.
- At most 200 submissions are fetched per platform per sync and 300 are retained per user, so analytics reflect **recent** performance rather than complete history.
- The AI Coach requires an internet connection and a valid Mistral (or OpenRouter) API key.
- The project was developed by a single student within one academic semester (17 weeks).

4. System Modeling

System modeling provides the behavioural view of CodePulse. This chapter presents the Level-0 use case diagram, detailed descriptions of the five principal use cases, and activity diagrams for each core execution pipeline across the Spring Boot backend, the PostgreSQL persistence layer and the external integrations.

4.1 Use Case Modeling

4.1.1 Level-0 System Use Case Diagram

The diagram shows the interaction between the primary human actor (Authenticated User), the automated System Scheduler, the system boundary, and the external services (Competitive Programming APIs and the Mistral AI engine).

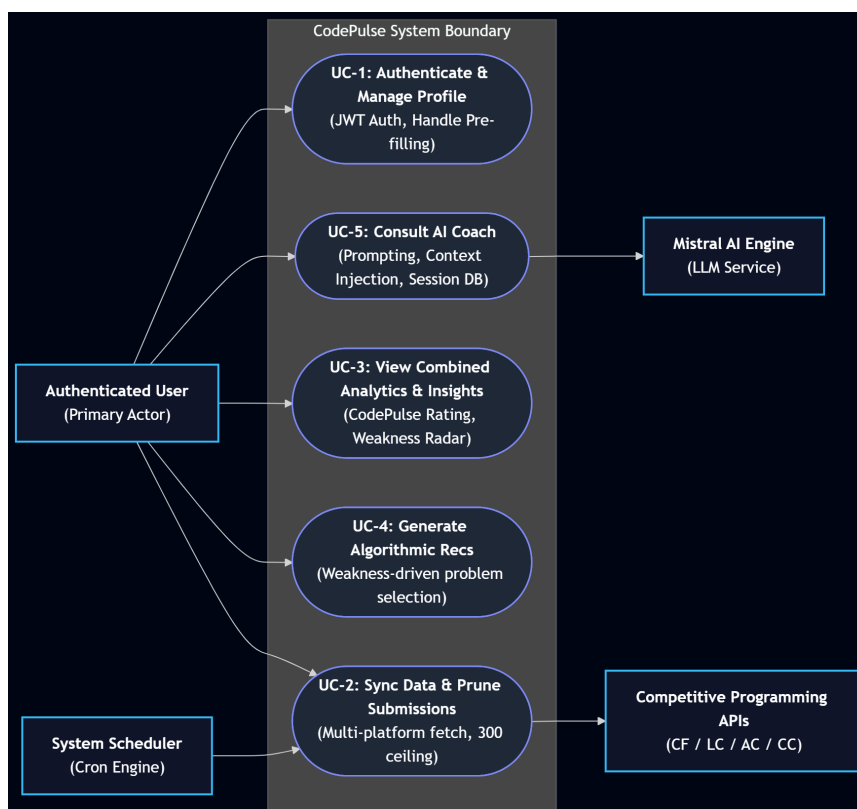


Figure 1: Level-0 Use Case Diagram

Table 12: Primary use cases

Use case	Summary
Register / Login	Create an account and authenticate; a 24-hour JWT is issued.
Manage Profile	Add or update platform handles and view connection status.
Sync Data	Fetch, deduplicate and store submissions from all connected platforms (manual or scheduled).
View Dashboard	See CodePulse Rating, streak, platform connections and recent submissions.
View Analytics	Explore charts, per-platform breakdown and hidden insights.
Get Curated Recommendations	Browse algorithmically selected problems; mark solved or dismiss.
Ask AI Coach	Enter a free-text goal and receive personalised AI-generated problems.
Administer Users	Administrators list and deactivate users (ROLE_ADMIN).

4.1.2 UC-1: Authentication and Profile Handle Configuration

Table 13: Use case description UC-1

Field	UC-1 Authentication & Profile Handle Configuration
Actor	Authenticated User
Trigger	User opens the Register / Login page or updates profile settings (/profile).
Preconditions	Backend is operational; PostgreSQL connection is active.
Main flow	1. User registers with email, username and password, or logs in. 2. Backend verifies credentials via BCrypt and issues a 24-hour signed JWT. 3. User enters or updates handles (Codeforces, LeetCode, AtCoder, CodeChef) in Profile. 4. Backend persists the handles in the <code>users</code> table. 5. System triggers an initial asynchronous sync for new handles.
Post-conditions	JWT stored in client state; handles saved and ready for cross-platform fetching.
Exceptions	Invalid credentials → 401 Unauthorized. Duplicate email / username → 400 / 409 with message.

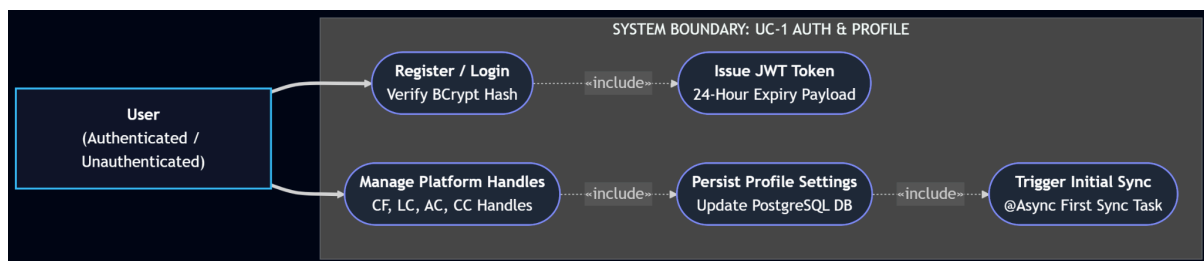


Figure 2: Use Case Diagram — User Authentication & Profile Configuration

4.1.3 UC-2: Multi-Platform Data Sync and Pruning

Table 14: Use case description UC-2

Field	UC-2 Multi-Platform Data Sync & Pruning
Actor	Authenticated User / Background Scheduler (<code>ScheduledSyncService</code>)
Trigger	User clicks "Sync All Platforms" (<code>POST /api/analytics/sync</code>) or the 6-hour cron fires.
Preconditions	User profile contains at least one connected platform handle.
Main flow	1. Parallel <code>@Async</code> requests are sent to the connected platform APIs. 2. Up to 200 recent submissions are fetched per platform. 3. Incoming submissions are deduplicated by <code>platformSubmissionId</code>. 4. New <code>Problem</code> and <code>Topic</code> records are created if absent. 5. <code>SubmissionPruningService</code> deletes the oldest submissions if the user has more than 300. 6. <code>lastSyncedAt</code> is updated on the user record.
Post-conditions	Database holds ≤ 300 recent submissions for the user; analytics are recalculated on demand.
Exceptions	Platform timeout / rate limit → error logged for that platform; remaining platforms continue.

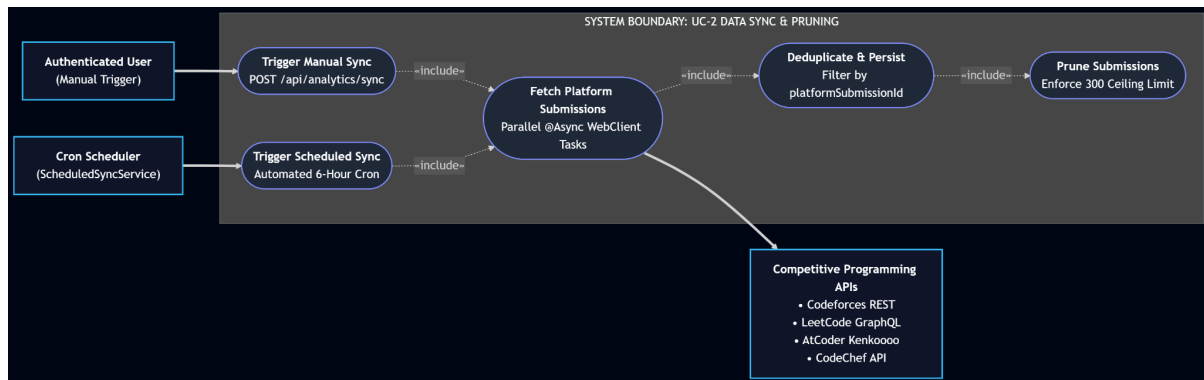


Figure 3: Use Case Diagram — Multi-Platform Data Sync & Pruning

4.1.4 UC-3: Analytics Computation and Hidden Insights

Table 15: Use case description UC-3

Field	UC-3 Analytics Computation & Hidden Insights Engine
Actor	Authenticated User
Trigger	User opens the Dashboard or Analytics page (<code>GET /api/analytics/me</code>).
Preconditions	Valid JWT; submissions have been synchronised.
Main flow	1. <code>AnalyticsServiceImpl</code> loads the user's submissions. 2. <code>CombinedRatingCalculator</code> computes per-platform ratings and the weighted CodePulse Rating. 3. Current streak, longest streak and active days are computed. 4. Topic failure rates produce strength / weakness distributions. 5. The Hidden Insights Engine derives 6–10 findings. 6. A <code>PerformanceAnalyticsResponse</code> DTO is returned to the frontend.
Post-conditions	Charts, rating, streak statistics and insights are rendered.
Exceptions	No submission history → zeroed DTO with empty-state flags.

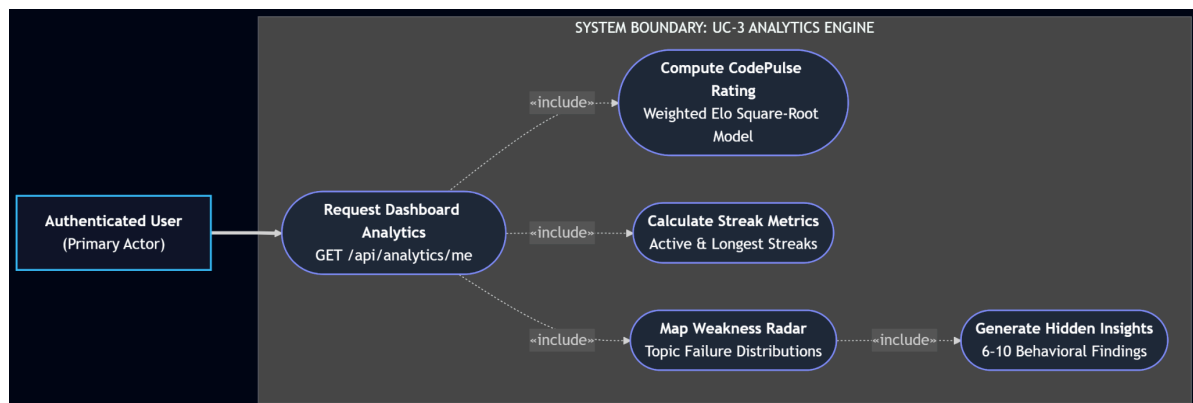


Figure 4: Use Case Diagram — Analytics Computation & Hidden Insights Engine

4.1.5 UC-4: Algorithmic Recommendation Generation and Feedback

Table 16: Use case description UC-4

Field	UC-4 Algorithmic Recommendation Generation & Feedback
Actor	Authenticated User
Trigger	User requests curated recommendations (<code>POST /api/recommendations/generate</code>).
Preconditions	Weakness scores can be computed from synchronised history.

Field	UC-4 Algorithmic Recommendation Generation & Feedback
Main flow	1. <code>RecommendationServiceImpl</code> identifies the top weakness topics. 2. The engine derives the user's difficulty bracket from the CodePulse Rating. 3. Unsolved problems in those topics within the bracket are queried. 4. Up to 10 curated problems are selected across all platforms. 5. Previous recommendations are deleted and the new ones persisted. 6. User marks items solved (<code>PATCH .../solved</code>) or dismissed (<code>PATCH .../dismiss</code>).
Post-conditions	Recommendation list updated; solved / dismissed flags persisted.
Exceptions	Insufficient problem catalogue → fall back to difficulty-matched problems.

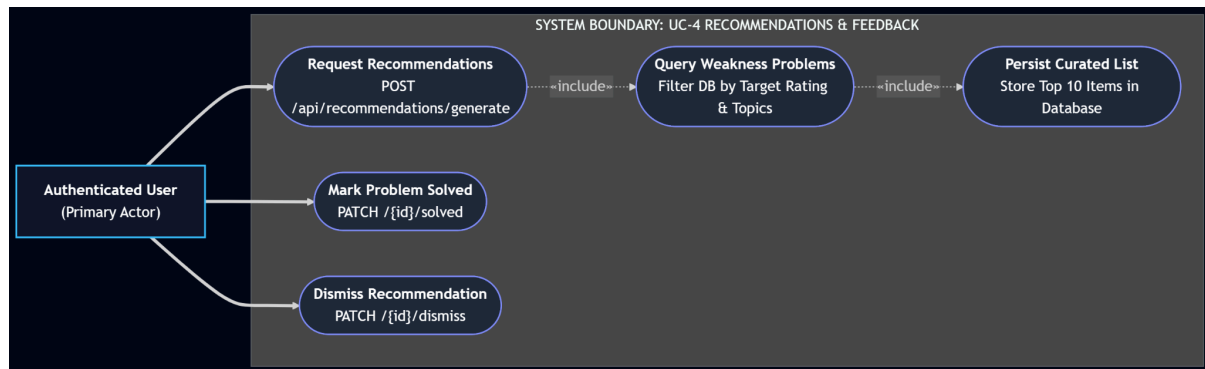


Figure 5: Use Case Diagram — Algorithmic Recommendation Generation & Feedback

4.1.6 UC-5: AI Coach Consultation and Session Persistence

Table 17: Use case description UC-5

Field	UC-5 AI Coach Consultation & Session Persistence
Actor	Authenticated User
Trigger	User submits a free-text practice prompt (<code>POST /api/ai/recommend</code>).
Preconditions	Mistral API key configured; user logged in.
Main flow	1. Context assembly: rating, per-platform stats, top 8 weak topics, top 5 strong topics, recent WA/TLE topics. 2. Prompt assembler merges the user prompt, context and a strict JSON schema. 3. Backend sends a request to the Mistral endpoint (<code>mistral-large-latest</code>). 4. A <code>JsonNode</code> parser validates the response structure. 5. The previous session is set inactive. 6. New session and items are saved. 7. Recommendations, coach insight and focus areas are returned.
Post-conditions	Session history stored; latest session auto-loads on page render.
Exceptions	Malformed JSON or API error → exception caught and logged; <code>buildError()</code> fallback; previous session retained.

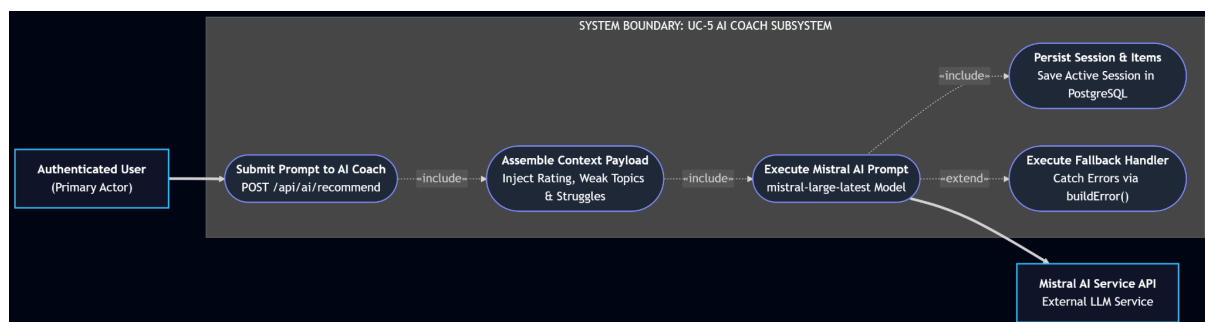
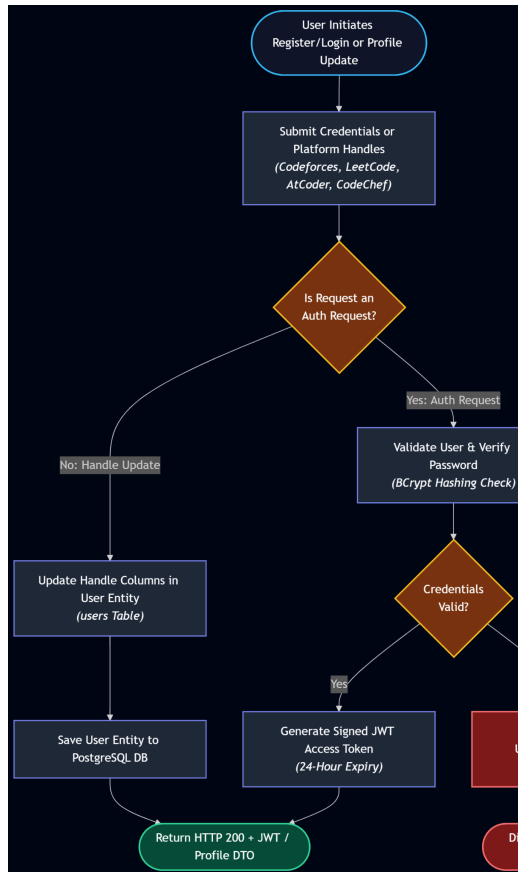


Figure 6: Use Case Diagram — AI Coach Consultation & Session Persistence

4.2 Activity Diagrams

Activity diagrams model the operational flows, branching decisions and state transitions of the five core pipelines. Each diagram is shown beside a step-by-step explanation.

4.2.1 AD-1: User Authentication and Profile Configuration

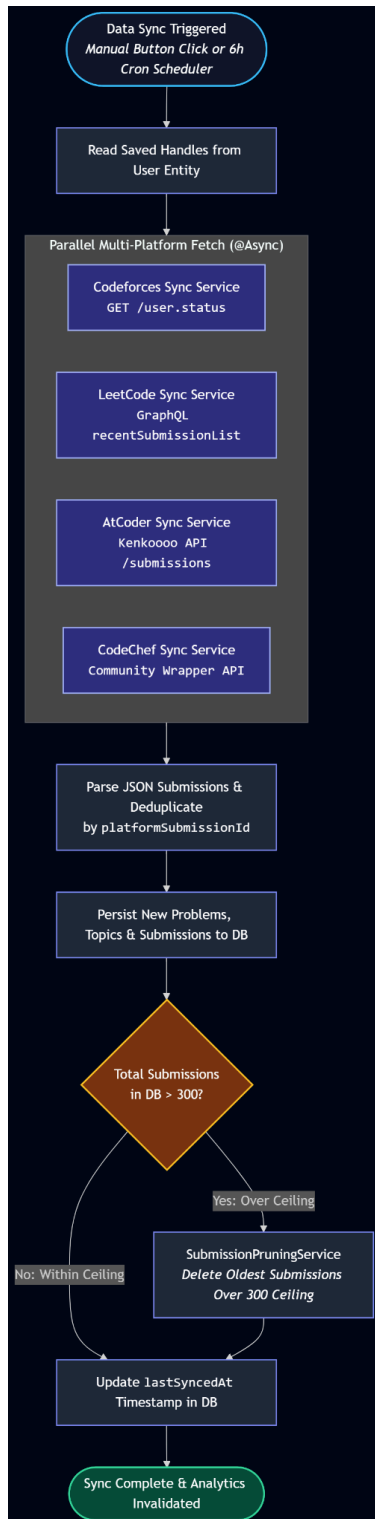


This flow captures registration, BCrypt password hashing, JWT issuance and persistence of platform handles.

1. The user submits credentials or profile data.
2. A decision node distinguishes **login**, **registration** and **profile update** requests.
3. For login the password is verified against the stored BCrypt hash; failure returns **401**.
4. For registration the system checks email / username uniqueness; a duplicate returns an error, otherwise the password is hashed and the user saved.
5. A signed 24-hour JWT is generated and returned.
6. Profile updates persist the four handle columns in `users` and trigger an asynchronous initial sync.

Figure 7: Activity Diagram — Authentication & Profile Configuration

4.2.2 AD-2: Multi-Platform Async Synchronisation and Pruning



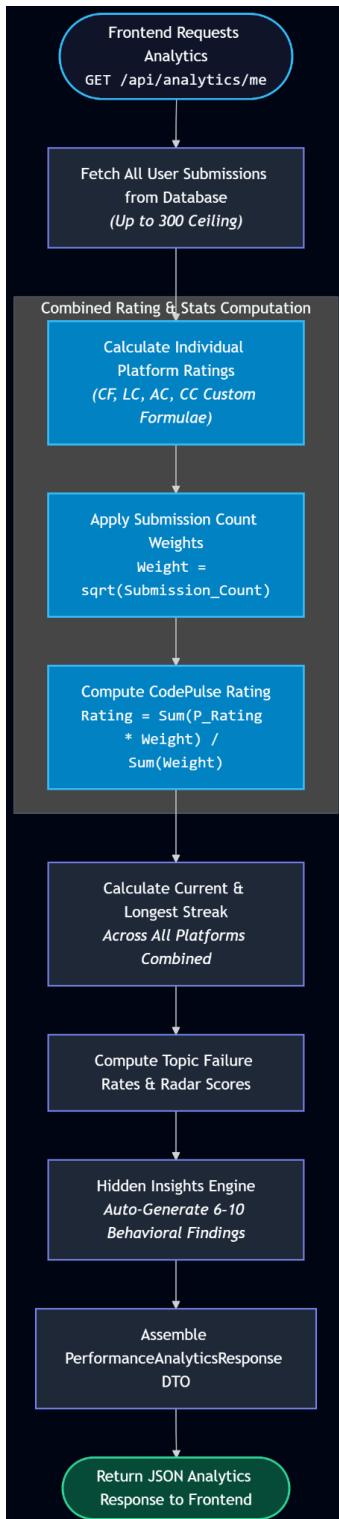
This flow captures asynchronous multi-platform fetching, deduplication, storage and the 300-submission ceiling.

1. Sync is initiated by a manual click or by the 6-hour cron job.
2. Saved handles are read from the `User` entity.
3. Four `@Async` services run **in parallel**: Codeforces `/user.status`, LeetCode `recentSubmissionList`, AtCoder `kenkoooo/submissions`, CodeChef community API.
4. Responses are parsed and deduplicated by `platformSubmissionId`.
5. New problems, topics and submissions are persisted.
6. If total submissions > 300, `SubmissionPruningService` deletes the oldest surplus rows.
7. `lastSyncedAt` is updated and cached analytics are invalidated.

A failure in one lane is logged and does not abort the other lanes (NFR-3).

Figure 8: Activity Diagram — Multi-Platform Async Synchronisation & Pruning

4.2.3 AD-3: Analytics Engine and CodePulse Rating

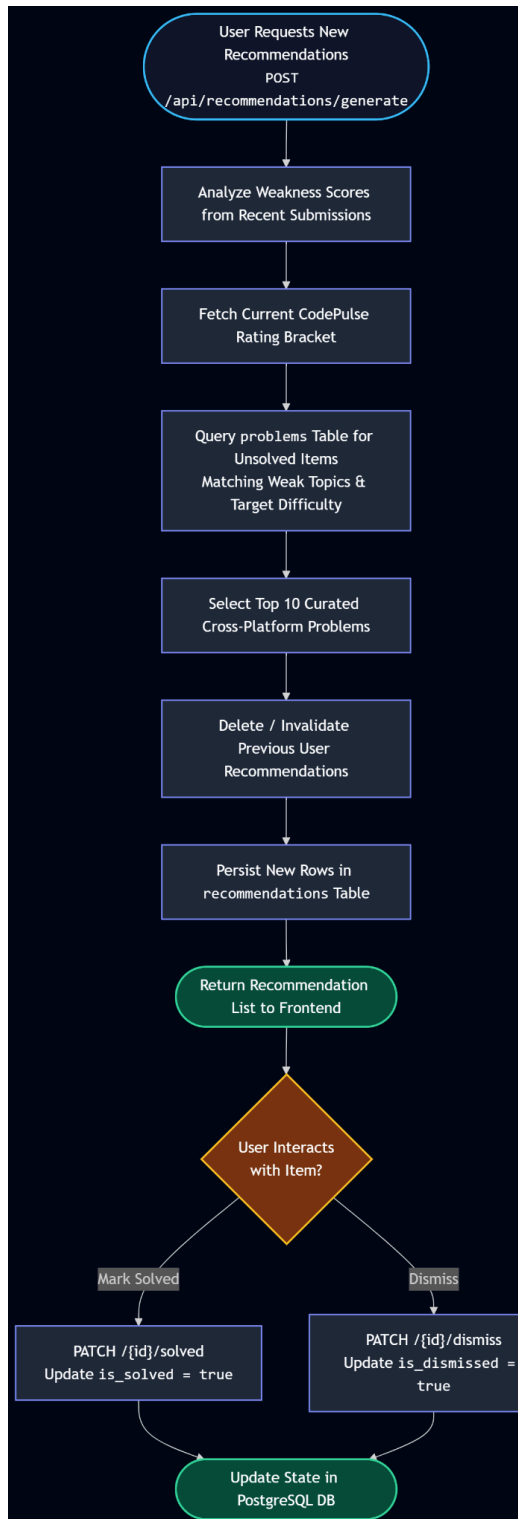


This flow details how an analytics request aggregates cross-platform data.

1. The frontend calls `GET /api/analytics/me`.
2. All submissions (≤ 300) are fetched with problems and topics in a single `JOIN FETCH` query.
3. Individual platform ratings are computed with the platform-specific formulas.
4. Weights $\sqrt{\text{submission count}}$ are applied and the CodePulse Rating is the weighted average, clipped to $[800, 3800]$.
5. Current and longest streaks are calculated across all platforms.
6. Topic failure rates and radar scores are computed.
7. The Hidden Insights Engine derives 6–10 findings.
8. The assembled `PerformanceAnalyticsResponse` is returned.

Figure 9: Activity Diagram — Analytics Engine & CodePulse Rating Pipeline

4.2.4 AD-4: Algorithmic Recommendation and Feedback

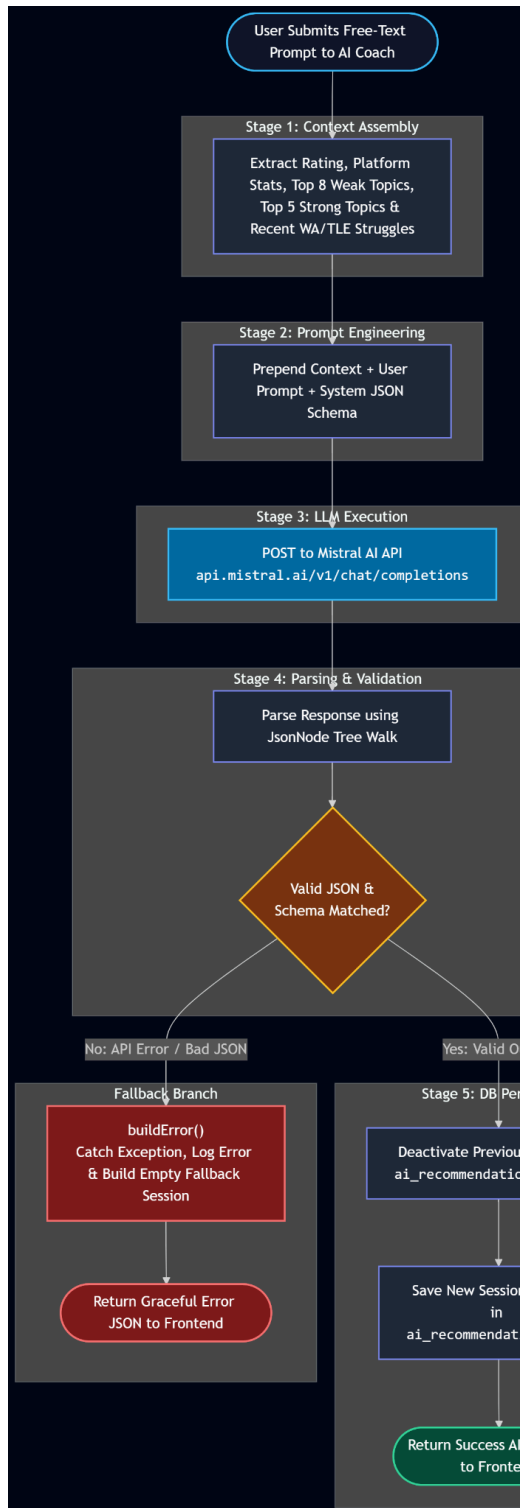


This flow illustrates weak-topic detection, rating-bracket querying, invalidation of old recommendations and user feedback.

1. The user requests new recommendations.
2. Weakness scores are derived from recent submissions.
3. The current rating bracket is determined.
4. The `problems` table is queried for **unsolved** items in weak topics and the target difficulty range.
5. Up to 10 problems are selected across platforms.
6. Previous recommendations are deleted; new rows are persisted.
7. When the user interacts, **Mark Solved** sets `is_solved = true` and **Dismiss** sets `is_dismissed = true`.

Figure 10: Activity Diagram — Algorithmic Recommendation & Feedback Pipeline

4.2.5 AD-5: AI Coach Prompt-to-Recommendation



This flow captures the full vertical pipeline from the user's prompt to a persisted session.

1. Stage 1 — context assembly (rating, platform stats, weak / strong topics, struggles).
2. Stage 2 — prompt assembly with a strict JSON schema.
3. Stage 3 — HTTP call to the Mistral chat-completions endpoint.
4. Stage 4 — `JsonNode` parsing and schema validation. Invalid output follows the **error branch**: exception caught, logged, `buildError()` returns a graceful fallback and the previous session is preserved.
5. Stage 5 — previous session deactivated; new session and items saved in one transaction; the result is returned to the UI.

Figure 11: Activity Diagram — AI Coach Prompt-to-Recommendation Pipeline

5. Data and Information Modeling

5.1 Database Schema

CodePulse uses PostgreSQL 15 with seven core tables plus one join table. All tables use auto-generated `BIGINT` primary keys with foreign-key constraints. The schema is generated and evolved by Hibernate (`ddl-auto=update`). Open-session-in-view is disabled, so every lazy association is loaded explicitly (see Section 6.5).

Table 18: Database tables

Table	Key columns	Purpose
<code>users</code>	<code>id</code> , <code>username</code> , <code>email</code> , <code>password_hash</code> , <code>full_name</code> , <code>avatar_url</code> , <code>codeforces_handle</code> , <code>leetcode_handle</code> , <code>atcoder_handle</code> , <code>codechef_handle</code> , <code>role</code> , <code>active</code> , <code>last_synced_at</code>	Authentication and the four platform handles
<code>problems</code>	<code>id</code> , <code>platform_id</code> , <code>platform</code> , <code>title</code> , <code>difficulty_rating</code> , <code>difficulty_label</code> , <code>problem_url</code>	Cross-platform problem catalogue
<code>topics</code>	<code>id</code> , <code>name</code> , <code>slug</code>	Algorithm topic tags (20 seeded on first boot)
<code>problem_topics</code>	<code>problem_id</code> (FK), <code>topic_id</code> (FK)	Many-to-many join between problems and topics
<code>submissions</code>	<code>id</code> , <code>user_id</code> (FK), <code>problem_id</code> (FK), <code>platform_submission_id</code> , <code>verdict</code> , <code>language</code> , <code>submitted_at</code> , <code>synced_at</code>	All synchronised submissions (pruned to 300 per user)
<code>recommendations</code>	<code>id</code> , <code>user_id</code> (FK), <code>problem_id</code> (FK), <code>reason</code> , <code>score</code> , <code>is_solved</code> , <code>is_dismissed</code>	Algorithmic curated recommendations
<code>ai_recommendation_sessions</code>	<code>id</code> , <code>user_id</code> (FK), <code>prompt</code> , <code>coach_insight</code> , <code>focus_areas_json</code> , <code>model_used</code> , <code>active</code> , <code>created_at</code>	One AI Coach consultation; one active per user
<code>ai_recommendation_items</code>	<code>id</code> , <code>session_id</code> (FK), <code>title</code> , <code>platform</code> , <code>url</code> , <code>difficulty</code> , <code>estimated_rating</code> , <code>topics_json</code> , <code>reason</code> , <code>time_estimate</code> , <code>is_solved</code> , <code>is_dismissed</code>	Individual problems within an AI session

5.2 Entity Details

Table 19: Selected entity attributes

Entity	Attribute	Type	Constraint / note
User	<code>id</code>	<code>BIGINT</code>	PK, generated
	<code>username</code> , <code>email</code>	<code>VARCHAR</code>	UNIQUE, NOT NULL
	<code>password_hash</code>	<code>VARCHAR</code>	BCrypt hash, NOT NULL
	<code>*_handle</code> (x4)	<code>VARCHAR</code>	Nullable; one per platform
	<code>role</code>	<code>ENUM</code>	<code>ROLE_USER</code> (default) / <code>ROLE_ADMIN</code>
	<code>last_synced_at</code>	<code>TIMESTAMP</code>	Updated after each successful sync
Problem	<code>platform</code> + <code>platform_id</code>	<code>VARCHAR</code>	Logical natural key (e.g. CF + 1850A)
	<code>difficulty_rating</code>	<code>INT</code>	Numeric where the platform provides one
	<code>difficulty_label</code>	<code>VARCHAR</code>	Beginner / Easy / Medium / Hard / Expert
Submission	<code>platform_submission_id</code>	<code>VARCHAR</code>	Used for deduplication
	<code>verdict</code>	<code>VARCHAR</code>	<code>ACCEPTED</code> , <code>WRONG_ANSWER</code> , <code>TIME_LIMIT_EXCEEDED</code> , ...
	<code>submitted_at</code>	<code>TIMESTAMP</code>	Basis for streaks, heatmap and pruning order

Entity	Attribute	Type	Constraint / note
Recommendation	score	DOUBLE	Priority score shown as a progress bar
AI session	active	BOOLEAN	Exactly one active session per user
AI item	topics_json	TEXT	JSON array serialised as text

5.3 ER Diagram

Key cardinalities of the model:

- **User 1 — N Submission; Submission N — 1 Problem.**
- **Problem N — M Topic** realised through `problem_topics`.
- **User 1 — N Recommendation**, each referencing exactly one Problem.
- **User 1 — N AiRecommendationSession**, and **AiRecommendationSession 1 — N AiRecommendationItem**.

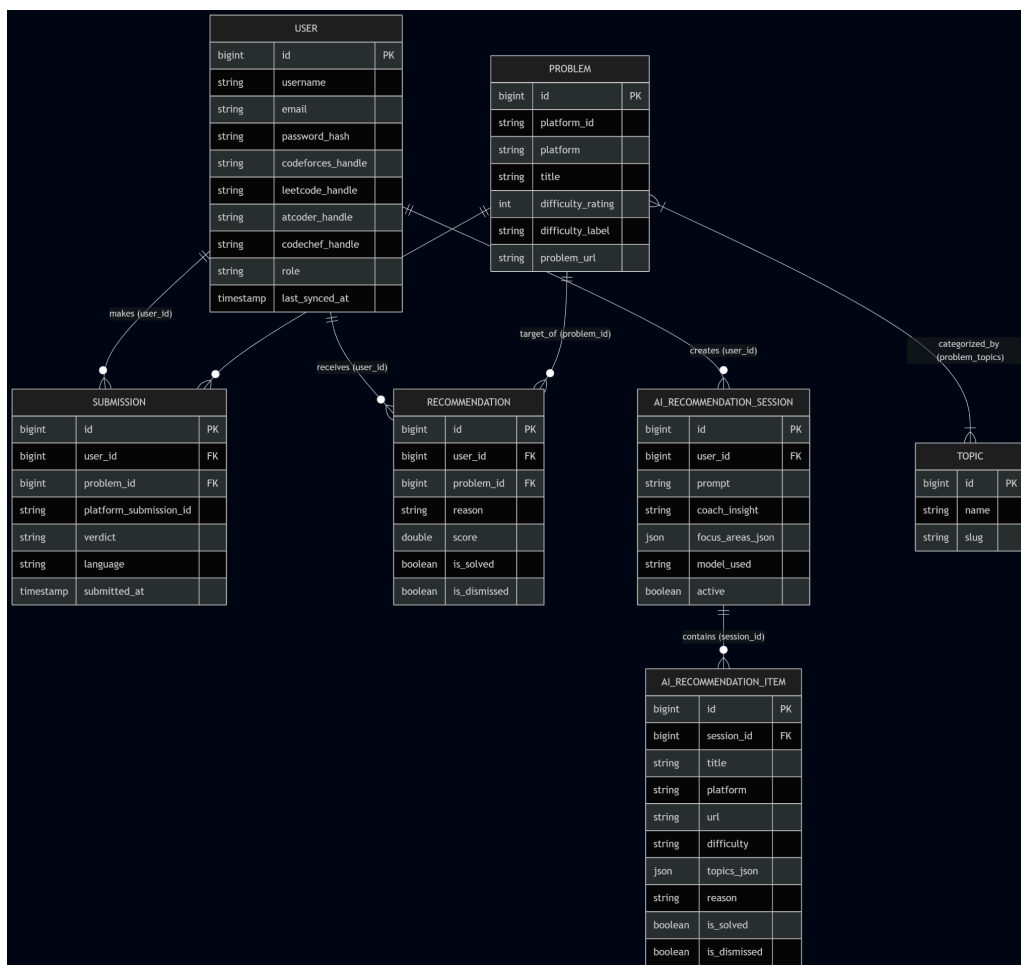


Figure 12: Entity–Relationship Diagram

5.4 Data Integrity and Retention Rules

- **Uniqueness:** a submission is identified by `(platform, platform_submission_id)`; repeated syncs never create duplicates (FR-6).
- **Bounded growth:** a user retains at most 300 submissions; the oldest by `submitted_at` are bulk-deleted after each sync (FR-7).
- **Replace-on-generate:** generating curated recommendations deletes the previous set for that user (FR-15).

- **Single active AI session:** creating a session first deactivates all earlier sessions for the user (FR-21).
- **Soft deactivation:** administrators deactivate users rather than deleting rows, preserving referential integrity.

6. Software Design — Component Level

This chapter presents the component-level design of CodePulse: the architecture, the design classes of the problem domain, the persistent data sources, behavioural representations of key components, the deployment view, the core algorithms, and a record of the alternatives that were considered and the refactorings applied.

6.1 Architectural Overview

CodePulse follows a clean **3-tier architecture** with an additional AI-integration layer and an asynchronous synchronisation layer. Separation of concerns keeps presentation, business logic and persistence independently testable and replaceable.

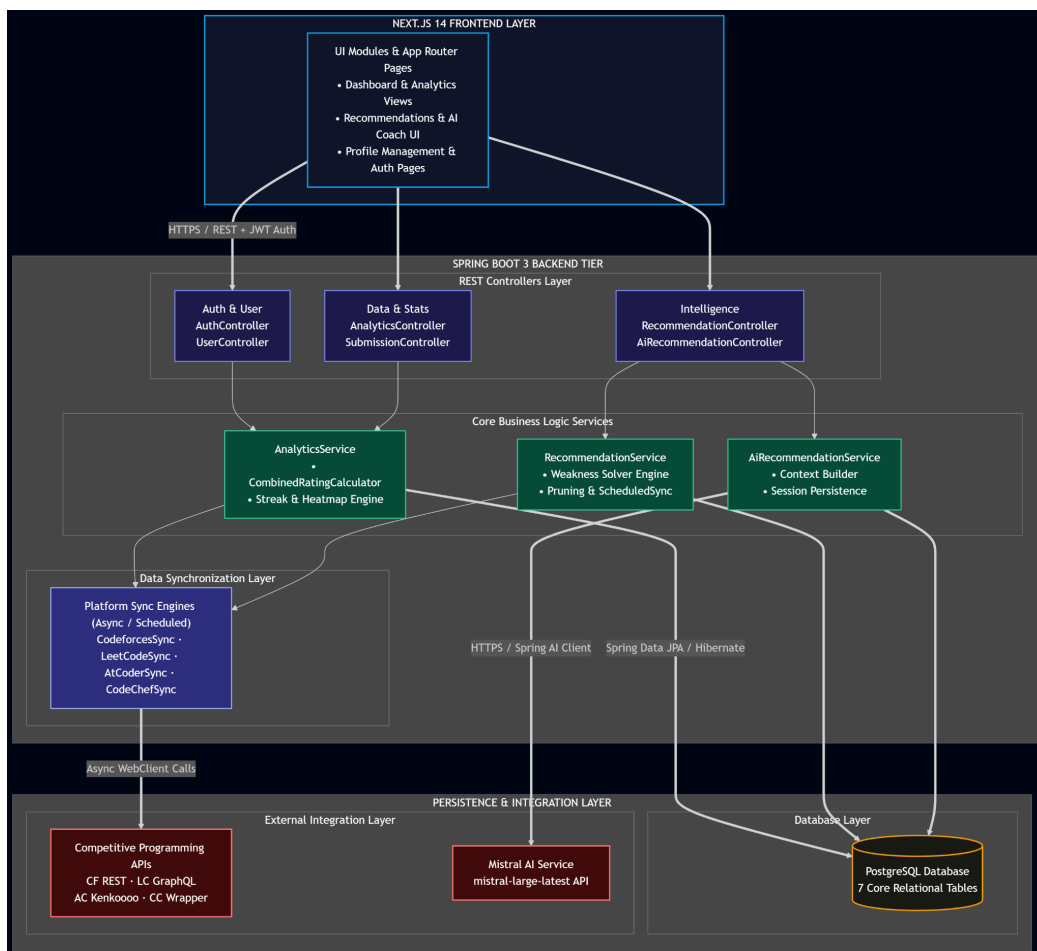


Figure 13: Component Diagram

Table 20: Layers and responsibilities

Tier / layer	Technology	Responsibility
Presentation	Next.js 14 (App Router), React, Tailwind CSS, Recharts, Framer Motion, Zustand, TanStack Query	Routing, forms, charts, client-side hidden-insights engine, JWT storage
Application — API	Spring MVC controllers (Auth, User, Analytics, Recommendation, AI, Submission, Admin)	HTTP contract, validation, DTO mapping, error translation
Application — business	Services: Analytics, Recommendation, AIRecommendation, Auth, User, Pruning	Rating, streaks, weakness scoring, recommendation selection, AI orchestration

Tier / layer	Technology	Responsibility
Application — integration	<code>CodeforcesDataService</code> , <code>LeetcodeDataService</code> , <code>AtCoderDataService</code> , <code>CodeChefDataService</code> , <code>DirectMistralService</code> , <code>OpenRouterService</code>	Outbound calls to third-party APIs; isolated per provider
Security	Spring Security, <code>JwtAuthenticationFilter</code> , <code>JwtUtil</code> , <code>UserDetailsServiceImpl</code>	Stateless authentication and role-based authorisation
Data	Spring Data JPA / Hibernate, PostgreSQL 15	Persistence, queries with <code>JOIN FETCH</code> , bulk delete for pruning

6.2 Package Structure

The backend is organised by technical layer under `com.codepulse`:

```

com.codepulse
├── config                DataSeeder (20 topics), SecurityConfig, WebClientConfig,
RestTemplate beans
├── controller            Auth, User, Analytics, Recommendation, AiRecommendation,
Submission, Admin
├── dto                   request/ (Login, Register, UpdateProfile, AiPrompt)
                           response/ (ApiResponse, AuthResponse, PerformanceAnalytics,
Recommendation, UserProfile, AiSession)
├── entity                User, Problem, Topic, Submission, Recommendation,
                           AiRecommendationSession, AiRecommendationItem
├── exception             BadRequest, ResourceNotFound, GlobalExceptionHandler
├── repository            one Spring Data repository per entity
├── security              JwtAuthenticationFilter, JwtUtil, UserDetailsServiceImpl,
UserPrincipal
├── service               interfaces + impl/ (Analytics, Auth, Recommendation, User,
AiRecommendation,
ScheduledSync,
CombinedRatingCalculator, DirectMistralService,
OpenRouterService)

```

6.3 Design Classes of the Problem Domain

The domain model identifies the persistent entity classes and their relationships. Each entity maps one-to-one to a table described in Chapter 5.

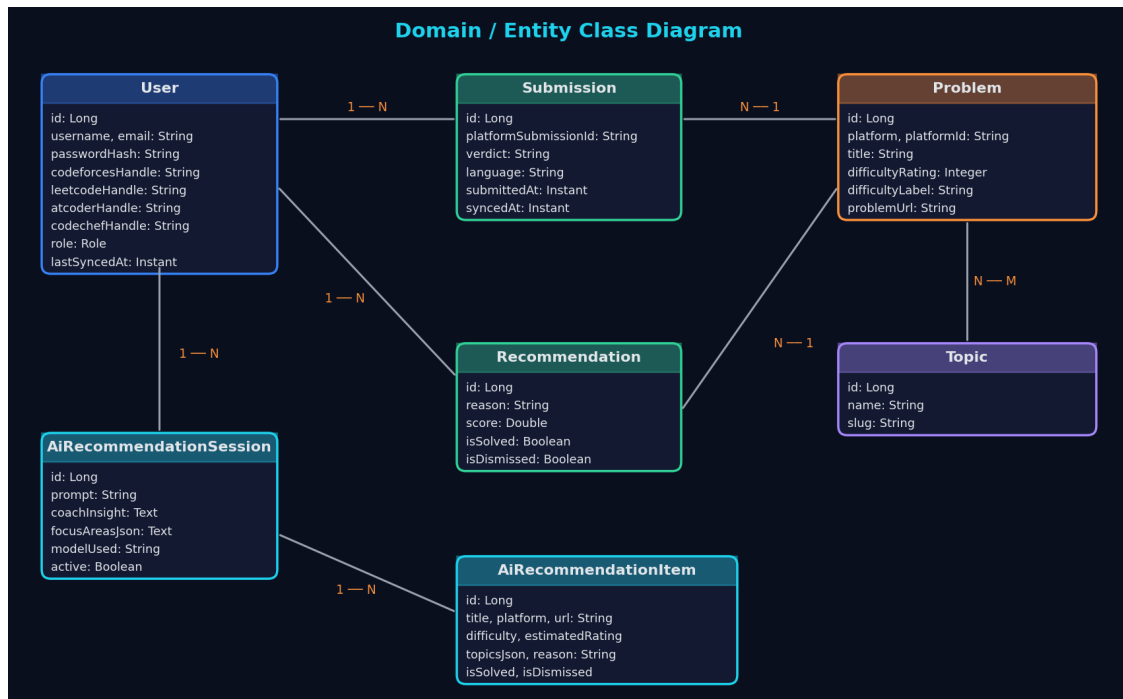


Figure 14: Domain (entity) class diagram

Table 21: Domain classes and responsibilities

Class	Responsibility	Key attributes / methods
User	Identity, credentials, role and the four platform handles	username, email, passwordHash, *Handle, role, lastSyncedAt
Problem	A problem on any platform with difficulty and topics	platform, platformId, title, difficultyRating, difficultyLabel, problemUrl
Topic	Algorithm tag; many-to-many with Problem	name, slug
Submission	One attempt by a user on a problem	verdict, language, submittedAt, syncedAt
Recommendation	Curated problem for a user with reason and score	score, isSolved, isDismissed
AiRecommendationSession	One AI coaching consultation	prompt, coachInsight, focusAreasJson, modelUsed, active
AiRecommendationItem	One recommended problem inside a session	title, platform, url, difficulty, topicsJson, reason

6.4 Service-Layer Design Classes

Business logic lives in stateless Spring services. Interfaces (`AnalyticsService`, `RecommendationService`, `AuthService`, `UserService`) decouple controllers from implementations, which makes mocking in unit tests straightforward.

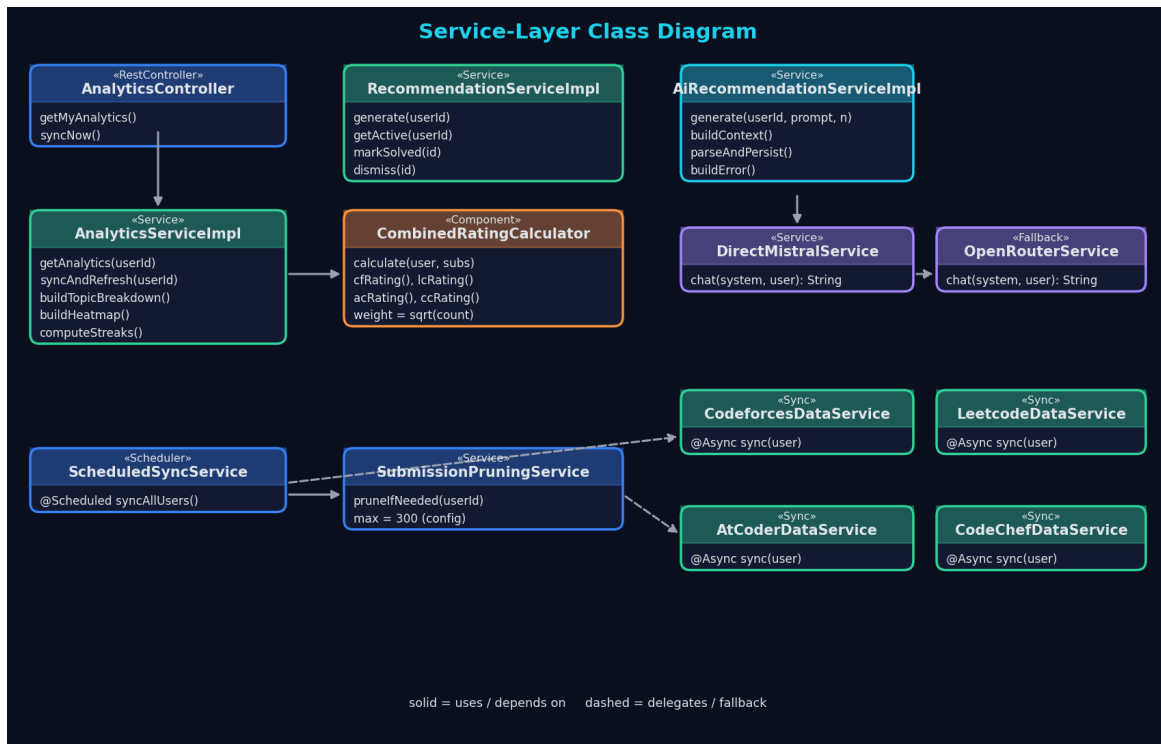


Figure 15: Service-layer class diagram

Table 22: Service-layer classes

Class	Responsibility	Key methods
<code>AnalyticsServiceImpl</code>	Computes all combined metrics, streaks, topic breakdown, heatmap, verdict distribution	<code>getAnalytics()</code> , <code>syncAndRefresh()</code>
<code>CombinedRatingCalculator</code>	Weighted Elo-like rating across four platforms	<code>calculate(user, submissions)</code>
<code>RecommendationServiceImpl</code>	Weakness-based algorithmic problem selection and feedback	<code>generate()</code> , <code>getActive()</code> , <code>markSolved()</code> , <code>dismiss()</code>
<code>AiRecommendationServiceImpl</code>	Builds context, calls the LLM, parses and persists the session	<code>generate()</code> , <code>buildContext()</code> , <code>parseAndPersist()</code> , <code>buildError()</code>
<code>DirectMistralService</code>	Calls Mistral REST API through <code>RestTemplate</code> , returns text	<code>chat(systemPrompt, userMessage)</code>
<code>OpenRouterService</code>	Fallback LLM provider through the same contract	<code>chat(systemPrompt, userMessage)</code>
<code>SubmissionPruningService</code>	Enforces the per-user submission ceiling	<code>pruneIfNeeded(userId)</code>
<code>ScheduledSyncService</code>	Triggers sync for all active users every 6 hours	<code>@Scheduled syncAllUsers()</code>
<code>CodeforcesDataService</code>	Codeforces <code>/user.status</code> synchronisation	<code>@Async syncUserSubmissions(user)</code>
<code>LeetcodeDataService</code>	LeetCode GraphQL <code>recentSubmissionList</code>	<code>@Async syncUserSubmissions(user)</code>
<code>AtCoderDataService</code>	AtCoder <code>kenkoooo</code> submissions	<code>@Async syncUserSubmissions(user)</code>
<code>CodeChefDataService</code>	CodeChef community API	<code>@Async syncUserSubmissions(user)</code>
<code>JwtUtil / JwtAuthenticationFilter</code>	Create, validate and apply JWT on each request	<code>generateToken()</code> , <code>validate()</code> , <code>doFilterInternal()</code>

6.5 Persistent Data Sources

6.5.1 Databases

- **Primary store:** PostgreSQL 15. All eight tables of Section 5.1 are JPA-managed entities; Hibernate `ddl-auto=update` creates or alters tables at start-up.
- **Seed data:** the `DataSeeder` bean inserts 20 algorithm topics (Dynamic Programming, Graphs, Binary Search, Greedy, Trees, ...) on first boot if the `topics` table is empty.
- **External read-only sources:** Codeforces, LeetCode, AtCoder and CodeChef APIs are treated as remote data sources; nothing is written back to them.
- **Files:** the system persists no user files. Configuration is read from `application.properties` and environment variables.

6.5.2 Repository Classes

Table 23: Repository queries

Repository	Notable queries
<code>UserRepository</code>	<code>findByEmail</code> , <code>findByUsername</code> , <code>existsByEmail</code> , <code>existsByUsername</code> , <code>findAllByActiveTrue</code>
<code>SubmissionRepository</code>	<code>findById</code> and <code>findRecentById</code> using <code>SELECT DISTINCT s FROM Submission s JOIN FETCH s.problem p LEFT JOIN FETCH p.topics</code> ; existence check by platform submission id; bulk delete of oldest rows
<code>ProblemRepository</code>	lookup by platform + platformId; unsolved problems in topic set within a difficulty range
<code>TopicRepository</code>	<code>findBySlug</code> ; seeding check
<code>RecommendationRepository</code>	<code>findActiveById</code> with JOIN FETCH on problem and topics; delete by user
<code>AIRecommendationSessionRepository / ItemRepository</code>	latest active session by user; deactivate all by user; <code>saveAll</code> for items

6.5.3 Lazy-Loading Strategy

With `spring.jpa.open-in-view=false`, the persistence session closes as soon as a repository method returns. Accessing a lazy association afterwards raises `LazyInitializationException`. The design addresses this in two layers: (1) repository queries load `Problem` and `Topic` data with `JOIN FETCH` in the same SQL round-trip, and (2) service and controller methods that traverse associations are annotated `@Transactional(readOnly = true)` as a safety net.

6.6 Behavioural Representations

6.6.1 Sequence — Analytics Retrieval

The sequence below shows how the analytics component collaborates with the security filter, repository and rating calculator.

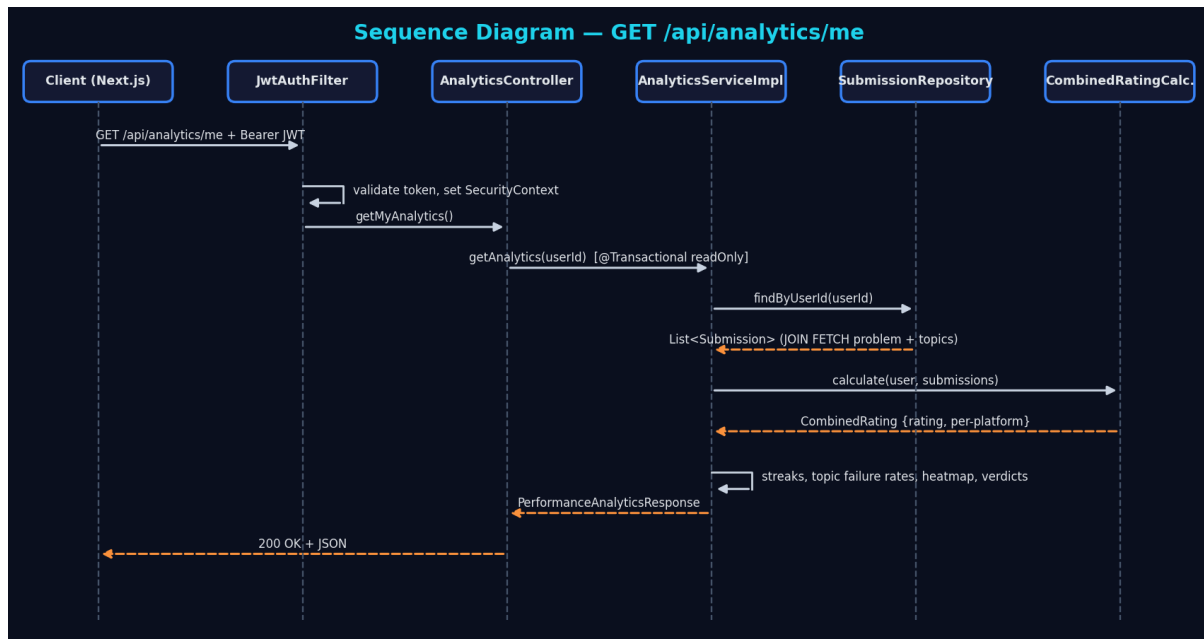


Figure 16: Sequence diagram — GET /api/analytics/me

6.6.2 Sequence — AI Recommendation

The AI component orchestrates context building, the LLM call, parsing and persistence. Failure at the parsing stage diverts to the `buildError()` fallback.

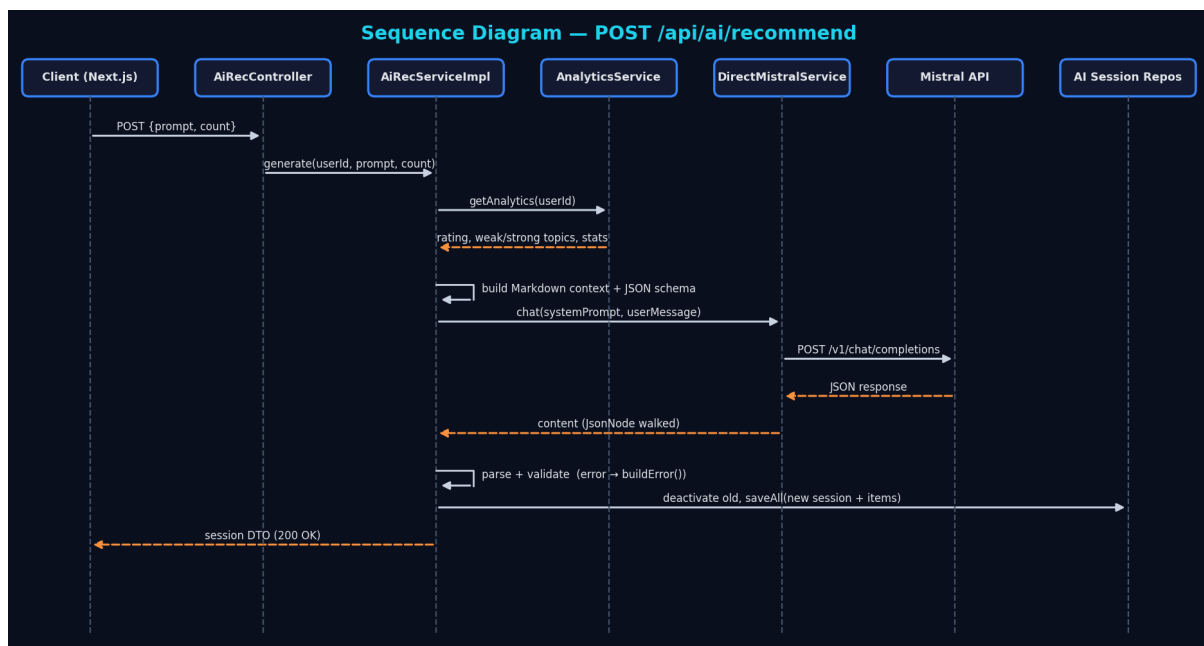


Figure 17: Sequence diagram — POST /api/ai/recommend

6.6.3 Lifecycle States of a Recommendation

Both curated recommendations and AI items share the same lifecycle:

Table 24: Recommendation lifecycle

State	Entry event	Exit events	Visible to user
Active	Generated by engine / AI	Solve, dismiss, regenerate	Yes
Solved	PATCH .../solved	Regenerate (replaced)	No (removed from active list)
Dismissed	PATCH .../dismiss	Regenerate (replaced)	No

State	Entry event	Exit events	Visible to user
Replaced / inactive	New generation request	—	No (AI session <code>active = false</code>)

6.6.4 Behaviour of the Synchronisation Component

`ScheduledSyncService` iterates active users and invokes the four `@Async` services. Each service executes the same template: (1) read handle, (2) call remote API with a timeout, (3) map the response to `Problem`, `Topic` and `Submission` objects, (4) skip any submission whose `platformSubmissionId` already exists, (5) save the remainder. Exceptions are caught per platform, logged with the platform name, and never propagate to the scheduler. After all lanes complete, `SubmissionPruningService.pruneIfNeeded()` runs once per user.

6.7 Deployment Diagram

The deployment view elaborates the physical nodes, their runtime technology and communication paths. In development all three nodes run on one machine; in a production-like setup they can be separated without code changes because they communicate only through HTTP and JDBC.

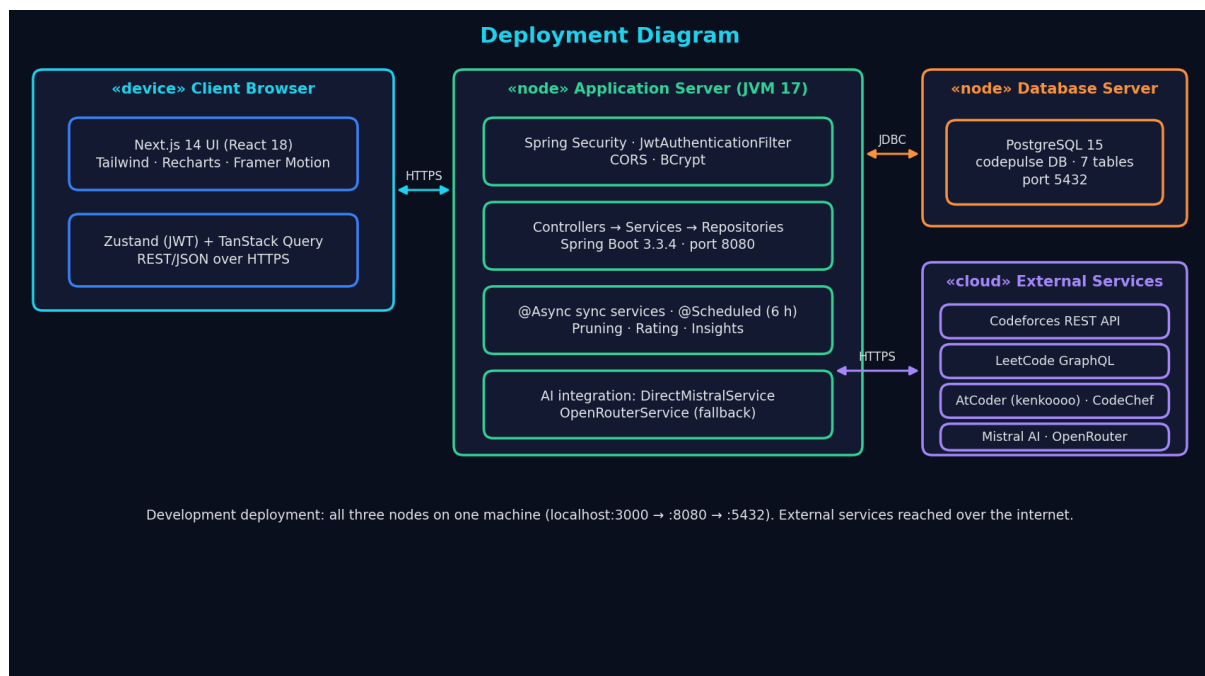


Figure 18: Deployment diagram

Table 25: Deployment nodes

Node	Runtime	Port	Protocol
Client browser	Next.js 14 application (dev server or built output)	3000	HTTPS / JSON to backend
Application server	JVM 17 running the Spring Boot 3.3.4 executable JAR	8080	REST; JDBC to DB; HTTPS to APIs
Database server	PostgreSQL 15, database <code>codepulse</code>	5432	JDBC
External services	Codeforces, LeetCode, AtCoder (kenkoooo), CodeChef, Mistral AI, OpenRouter	443	HTTPS

6.8 Core Algorithms

6.8.1 CodePulse Rating

A single Elo-like number is computed from all connected platforms. Per-platform ratings are derived as follows:

Table 26: Per-platform rating formulas

Platform	Rating formula	Cap
Codeforces	Actual rating from <code>/user.rating</code> ; otherwise estimated from the average difficulty of solved problems	—
LeetCode	$1200 + (\text{easy} \times 2) + (\text{medium} \times 9) + (\text{hard} \times 22) + \text{acceptance-rate bonus}$	3200
AtCoder	Average difficulty of solved problems $\times 1.05$	—
CodeChef	$1000 + (\text{easy} \times 3) + (\text{medium} \times 10) + (\text{hard} \times 20) + (\text{expert} \times 35)$	3000

```
weight(platform) = sqrt(submissionCount(platform)) // diminishing returns
CodePulse Rating = SUM(rating_p * weight_p) / SUM(weight_p)
CodePulse Rating = clamp(CodePulse Rating, 800, 3800)
```

Using the square root of the submission count prevents a platform with very many submissions from dominating the result while still rewarding sustained activity. Platforms with no activity contribute zero weight, so a user with only one connected platform still receives a meaningful rating.

6.8.2 Topic Weakness Score

```
for each topic t:
    attempts(t) = number of submissions on problems tagged t
    failures(t) = attempts(t) - accepted(t)
    weakness(t) = failures(t) / attempts(t) // 0.0 strong ... 1.0 weakest
weakTopics = topics sorted by weakness desc (min. attempts threshold applied)
strongTopics = topics sorted by weakness asc and accepted(t) high
```

6.8.3 Streak Calculation

Submission timestamps from all platforms are reduced to a set of distinct calendar days. The **current streak** counts consecutive days ending today (or yesterday); the **longest streak** is the maximum run in the set; **total active days** is the set size. Because platforms are merged first, two Codeforces submissions on day 1 and one LeetCode submission on day 2 yield a streak of 2.

6.8.4 Curated Recommendation Selection

```
weak = top weakness topics (failure rate > 30%)
rating = estimated user rating (CodePulse Rating)
range = difficulty band slightly above the estimated rating // stretch zone
cand = problems WHERE topic IN weak AND difficulty IN range AND id NOT IN
solved(user)
picks = rank cand by (weakness(topic), closeness to rating); take up to 10
if |picks| < 10: fill with difficulty-matched problems from any topic
delete previous recommendations(user); save picks with reason + score
```

The failure-rate threshold and difficulty band are configuration constants in the service and can be tuned without changing the architecture.

6.8.5 Submission Pruning

After every sync the service counts the user's submissions. If the count exceeds `codepulse.submissions.max-per-user` (default 300), the surplus oldest rows (ordered by `submitted_at` ascending) are removed using a single bulk `DELETE ... WHERE id IN (...)` statement. This keeps database size bounded and analytics always based on the latest activity.

6.8.6 Hidden Insights Engine

The insights engine runs in the frontend on the analytics response, with no extra backend calls. It evaluates a rule set and emits up to ten findings in five categories.

Table 27: *Hidden insights rule set*

Category	Rule (example)	Finding
Critical	Failure rate on a topic > 55%	Critical weakness with exact percentage
Critical	0 accepted submissions on a topic	Blind spot detected
Warning	Acceptance rate < 35%	Submit more carefully
Warning	> 75% of solves are Beginner / Easy	Comfort-zone alert
Warning	Streak broken after a best of ≥ 7 days	Restart today
Warning	Activity down > 50% versus previous 30 days	Activity drop
Success	Current streak ≥ 14 days	Warrior recognition
Success	Acceptance rate > 80%	Sharpshooter
Tip	< 6 distinct topics despite > 25 solved	Explore more topics
Pattern	Top three topics with ≥ 15 solves each	Signature strengths

6.9 Design Alternatives and Refactoring

Each component-level design representation was reviewed for alternatives and refactored where a better option emerged. The table records the principal decisions.

Table 28: *Design alternatives considered*

Component	Alternatives considered	Decision and rationale
LLM integration	(a) Spring AI SDK; (b) direct REST with <code>RestTemplate</code> + <code>JsonNode</code>	(b). Spring AI milestone releases (M4–M6) create an internal <code>ObjectMapper</code> that cannot be customised. When Mistral added <code>prompt_tokens_details</code> to its response, every SDK call failed with <code>UnrecognizedPropertyException</code> . Parsing into a <code>JsonNode</code> tree ignores unknown fields.
Lazy loading	(a) Open-session-in-view; (b) <code>JOIN FETCH</code> + <code>@Transactional(readonly)</code>	(b). OSIV hides N+1 queries and holds connections during view rendering. Explicit fetching gives one SQL round-trip per request and predictable performance.
Platform sync	(a) One monolithic sync class; (b) one service per platform	(b). Failures are isolated, platforms run in parallel and adding a platform needs only one new class (NFR-6).
Rating weight	(a) Arithmetic mean; (b) weight by submission count; (c) weight by $\sqrt{\text{count}}$	(c). Mean ignores effort; raw count lets one platform dominate; $\sqrt{}$ gives diminishing returns.
Data growth	(a) Keep all history; (b) cap per user	(b). A 300-row cap bounds storage and keeps analytics responsive (NFR-1, NFR-4).
Insights engine	(a) Backend computation; (b) client-side rules	(b). Rules need only the analytics response; client-side evaluation avoids extra API calls and keeps the backend focused on data.
LLM availability	(a) Single provider; (b) primary + OpenRouter fallback	(b). A one-property change switches model or provider with the same code path.
Sync trigger	(a) Only manual; (b) manual + 6-hour cron	(b). Fresh data without user action; manual option for immediacy.
Authentication	(a) Server sessions; (b) stateless JWT	(b). Fits the SPA frontend and keeps the backend horizontally scalable.

Refactorings applied during development: extraction of `CombinedRatingCalculator` out of `AnalyticsServiceImpl`; replacement of the SDK-based AI client by `DirectMistralService`;

introduction of `SubmissionPruningService` as a separate component; moving the `JOIN FETCH` logic into repository queries; and consolidating DTO construction into the service layer.

7. Interface Design

The interface was designed from the user-interface analysis of the primary users (competitive programmers and students) who want quick insight with minimal clicks. The visual language is a dark "cyberpunk" theme with neon accent colours, glassmorphism cards and animated transitions, implemented with Tailwind CSS and Framer Motion.

7.1 Design Principles

- **Glanceability:** the dashboard exposes four headline numbers (rating, solved, streak, acceptance) before any chart.
- **No duplication:** the Dashboard summarises; the Analytics page goes deep.
- **Feedback for every action:** skeleton loaders, spinners, toasts and empty states with a call to action (NFR-5).
- **Direct access to the source:** every problem title and platform handle is a link that opens the original page in a new tab.
- **Responsive layout:** sidebar navigation collapses on narrow screens.

7.2 Pages and Routes

Table 29: Pages, routes and UI objects

Route	Purpose	Key UI objects
<code>/auth/login</code>	Authenticate and receive a JWT	Email field, password field with visibility toggle, submit button, typing-animation terminal hint, link to register
<code>/auth/register</code>	Create an account (2-step wizard)	Step 1: username and full name. Step 2: email and password. Progress bar, Back / Next buttons
<code>/dashboard</code>	Combined overview	Four StatCards, StreakWidget, Platform Connection Grid, quick-action tiles, Recent Submissions timeline, Sync button
<code>/analytics</code>	Deep analysis	Insights panel, platform comparison bar chart, CF rating area chart, difficulty bar, weakness radar, verdict donut, topic coverage bars, activity heatmap
<code>/recommendations</code>	Curated problems	Platform filter tabs, problem cards with link, Solve / Dismiss buttons, score bar, Generate New button
<code>/ai-recommend</code>	AI Coach	Prompt textarea, example chips, count selector (4 / 6 / 8), Ask AI button, Coach Insight card, focus-area chips, AI cards
<code>/profile</code>	Account management	Avatar, connection-status badges, four handle inputs pre-filled from server, external profile links, Save button

7.3 User-Interface Objects and Actions

Table 30: Interface objects and operations

UI object	Description	Action (operation)	Resulting change
StatCard	Animated number tile with neon border and LIVE badge	On mount: CountUp from 0 to the value over 2 s	Number displayed
Platform grid cell	Handle, solved count and acceptance rate, or "Add handle →"	Click handle / link	Opens the platform profile in a new tab or navigates to <code>/profile</code>
Sync button	Triggers manual synchronisation	Click → <code>POST /api/analytics/sync</code>	Spinner shown; after ~4 s the query cache is invalidated and data refreshes

UI object	Description	Action (operation)	Resulting change
Submission row	Platform badge, title, topic chip, verdict badge, time-ago	Click title	Opens <code>problemUrl</code> in a new tab
Insight card	Coloured card by category with icon, title, text	Appear after data load	Staggered Framer Motion animation
AI prompt form	Textarea, chips, count selector, submit	Submit / Ctrl+Enter / click chip	POST; generating state; session replaced on success
Recommendation card	Platform badge, difficulty chip, title, topics, reason, score bar	Solve / Dismiss	PATCH call; card exits with animation
Profile form	Name, avatar URL, four handles	Save Changes → PUT <code>/api/users/me</code>	Toast confirmation; handles propagate to dashboard
Platform filter tabs	All / CF / LC / AC / CC	Click tab	List filtered client-side

7.4 Events and State Changes

Table 31: UI events and state transitions

Event (user action)	From state	To state
Open /dashboard	None	Loading skeleton
Profile + analytics loaded	Loading skeleton	Data loaded
Loaded but no handles saved	Loading skeleton	Empty state (prompt to add handles)
Click Sync All	Data loaded	Syncing (spinner)
Sync timeout elapses, cache invalidated	Syncing	Data loaded (refreshed)
Submit AI prompt	No session / Session displayed	Generating (animated dots)
AI response 200 with session	Generating	Session displayed
AI error	Generating	Error message with previous session retained
Click Solve / Dismiss	Session / list displayed	Item fades out; list updated
Token missing / expired (401)	Any page	Redirect to /auth/login

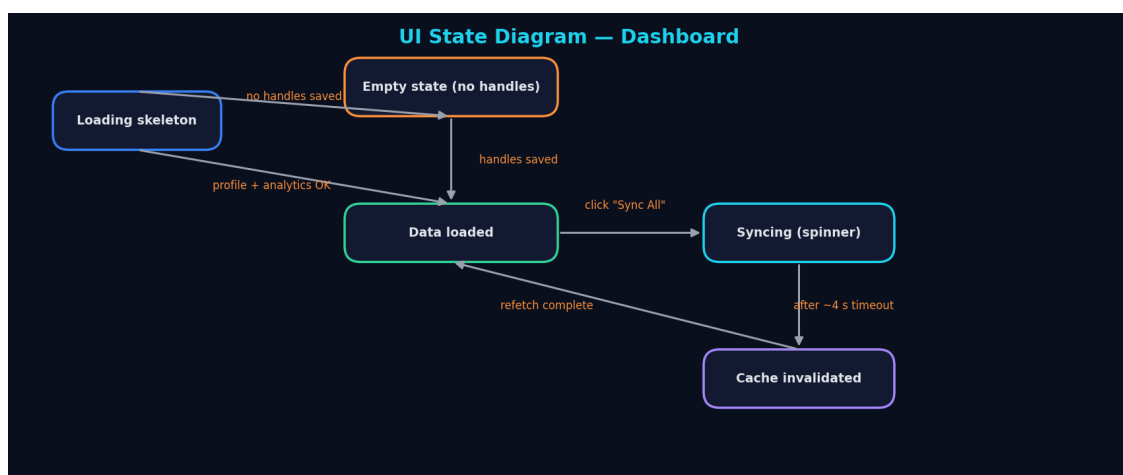


Figure 19: State diagram — Dashboard

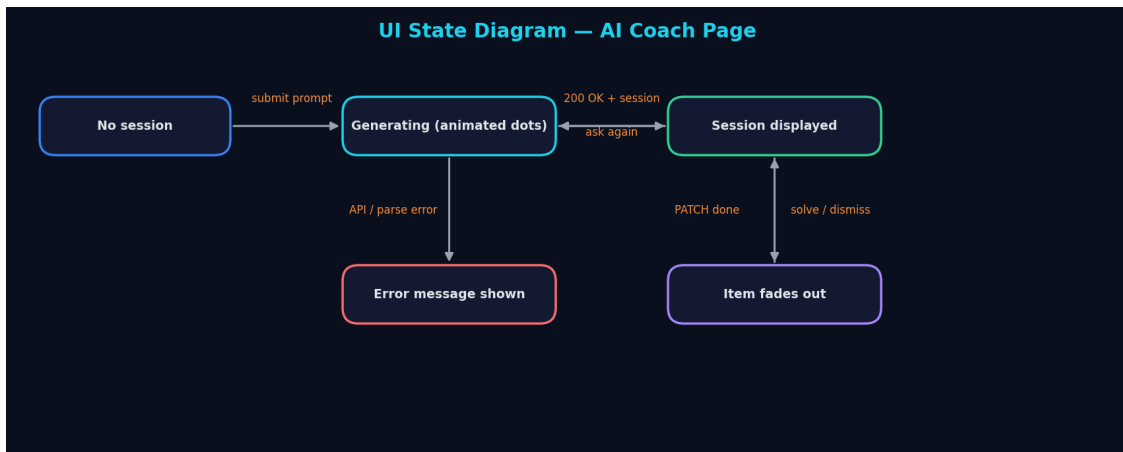


Figure 20: State diagram — AI Coach page

7.5 Screen Designs

The following screens depict each interface state as the end user sees it. Sample values are illustrative.

7.5.1 Authentication

Authentication Screens (Login and 2-step Registration)

LOGIN

Email

Password []

Sign In

> terminal hint (type animation)

No account? [Create Account](#)

REGISTER — Step 1 of 2

Username

Full Name

Next →

Step 2: Email + Password → "Launch"

Figure 21: Login and two-step registration screens

7.5.2 Dashboard



Figure 22: Dashboard screen

7.5.3 Analytics

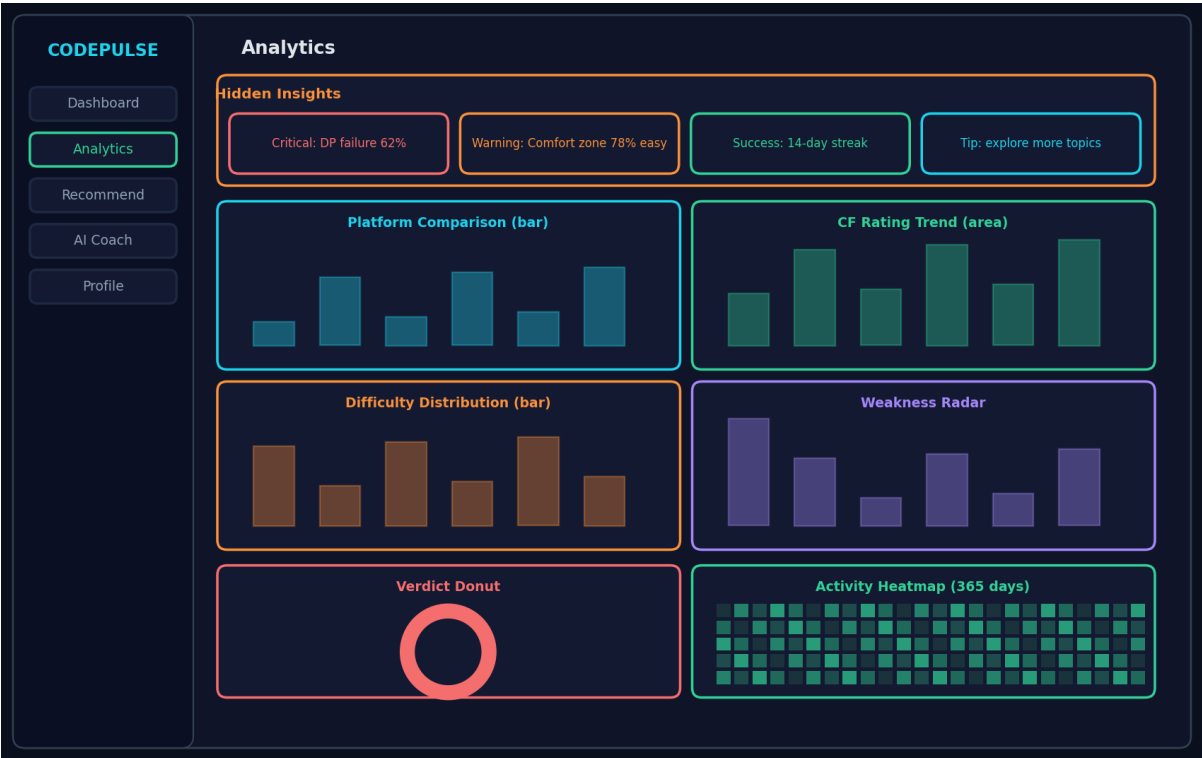


Figure 23: Analytics screen

7.5.4 Curated Recommendations



Figure 24: Recommendations screen

7.5.5 AI Coach

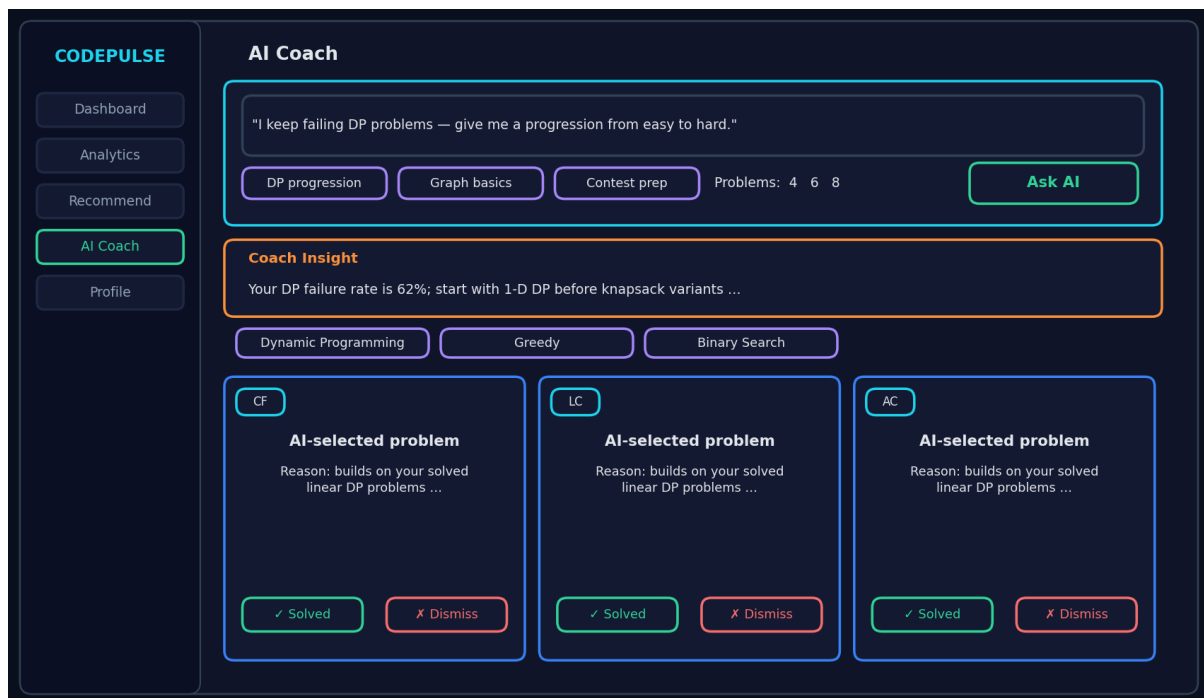


Figure 25: AI Coach screen

7.5.6 Profile

CODEPULSE

Dashboard

Analytics

Recommend

AI Coach

Profile

Profile

 **Abdus Salam Islam Badhon**
@badhon · USER

Codeforces handle ● connected

LeetCode username ● connected

AtCoder handle ○ not set

CodeChef username ● connected

Save Changes

Handles are pre-filled from the server on every load.

Figure 26: Profile screen

8. Implementation

8.1 Technology Stack

Table 32: *Technology stack*

Layer	Technology	Version / notes
Language (backend)	Java	17 (LTS)
Framework (backend)	Spring Boot	3.3.4 (spring-boot-starter-parent)
Web / validation	spring-boot-starter-web, spring-boot-starter-validation	REST controllers, bean validation
Security	Spring Security + jjwt	JWT HS256 (jjwt 0.11.5), BCrypt password encoder
ORM	Spring Data JPA / Hibernate	ddl-auto=update, open-in-view disabled
Database	PostgreSQL	15+
HTTP clients	Spring WebFlux WebClient; RestTemplate	WebClient for platform APIs; RestTemplate for LLM calls
AI (primary)	Mistral AI over direct HTTP	mistral-large-latest
AI (fallback)	OpenRouter	Any OpenAI-compatible model
Utilities	Lombok	Boilerplate reduction for entities and DTOs
Build	Maven	3.9+
Frontend	Next.js 14 (App Router), TypeScript, React	Server and client components
Styling / animation	Tailwind CSS, Framer Motion	Custom neon theme; page reveals and card exits
Charts	Recharts	Area, bar, pie / donut and radar charts
Client state	Zustand + TanStack Query v5	Auth store; server-state cache with invalidation
Tooling	Git / GitHub, Postman, IntelliJ / VS Code	Repository: github.com/Badhon1401/SPL-3

8.2 Backend Configuration

All environment-specific values live in `application.properties`; secrets should be supplied by environment variables in any shared deployment.

Table 33: *Key configuration properties*

Property	Purpose	Example
<code>spring.datasource.url</code>	JDBC URL of the PostgreSQL database	<code>jdbc:postgresql://localhost:5432/codepulse</code>
<code>spring.datasource.username / password</code>	Database credentials	<code>postgres / *secret*</code>
<code>spring.jpa.hibernate.ddl-auto</code>	Schema management	<code>update</code>
<code>spring.jpa.open-in-view</code>	Disable OSIV	<code>false</code>
<code>jwt.secret</code>	Base64-encoded 256-bit signing key	<code>openssl rand -base64 32</code>
<code>jwt.expiration</code>	Token lifetime in ms	<code>86400000</code> (24 h)
<code>spring.ai.mistralai.api-key</code>	Mistral API key	<code>*secret*</code>
<code>app.openrouter.api-key</code>	Optional fallback LLM key	<code>*secret*</code>
<code>codepulse.submissions.max-per-user</code>	Pruning ceiling	<code>300</code>

8.3 Security Implementation

Security is implemented with Spring Security in a **stateless** configuration. The flow on each request is:

1. The `JwtAuthenticationFilter` reads the `Authorization: Bearer <token>` header.
2. `JwtUtil` verifies the signature and expiry and extracts the user identity.
3. `UserDetailsServiceImpl` loads the user and wraps it in a `UserPrincipal` that carries the role.
4. The authenticated principal is placed in the `SecurityContext`.
5. `SecurityConfig` permits `/api/auth/**`, restricts `/api/admin/**` to `ROLE_ADMIN` and requires authentication for everything else; CORS is configured for the frontend origin.

Table 34: Security controls

Concern	Mechanism
Password storage	BCrypt hash; plaintext never stored or logged
Session management	Stateless; no server session; token in client store
Authorisation	Role claim; <code>@PreAuthorize</code> / URL rules for admin endpoints (403 for <code>ROLE_USER</code>)
Input validation	Bean Validation on request DTOs; <code>GlobalExceptionHandler</code> maps errors to <code>ApiResponse</code> with proper status codes
Secrets	JWT secret and API keys supplied via properties / environment, never committed
Data access	Parameterised JPA queries; no string-built SQL

8.4 Data Synchronisation Implementation

Each platform has its own `@Async` service. The common template is shown as pseudocode:

```
@Async syncUserSubmissions(user):
    handle = user.<platform>Handle ; if handle is blank: return
    try:
        raw = http GET/POST platform API (limit 200, timeout)
        for item in raw:
            if submissionRepo.exists(platform, item.id): continue          // dedup
(FR-6)
            problem = problemRepo.findOrCreate(platform, item.problem) // + topics
            submissionRepo.save(new Submission(user, problem, item.verdict,
item.time))
        catch (Exception e):
            log.error("Sync failed for {}: {}", platform, e)              // isolate
failure (NFR-3)
```

Platform-specific notes:

- **Codeforces:** `/user.status?handle=...&from=1&count=200`; verdicts such as `OK`, `WRONG_ANSWER` and `TIME_LIMIT_EXCEEDED` are normalised to the internal verdict names. Problem tags become topics; the numeric problem rating becomes `difficultyRating`. `/user.rating` supplies the rating trend.
- **LeetCode:** GraphQL query for recent submissions; the Easy / Medium / Hard label is stored as `difficultyLabel` because no numeric rating is exposed.
- **AtCoder:** kenkoooo community API for submissions and problem difficulty estimates.
- **CodeChef:** community wrapper API; difficulty is mapped to bands used by the rating formula.

`ScheduledSyncService` uses a Spring `@Scheduled` cron expression with a 6-hour period to synchronise all active users. The manual endpoint `POST /api/analytics/sync` calls the same services and returns immediately because the work runs asynchronously.

8.5 Analytics Implementation

`AnalyticsServiceImpl.getAnalytics()` is annotated `@Transactional(readOnly = true)`. It loads submissions once and derives every metric in memory, which is efficient for the bounded 300-row dataset. The returned `PerformanceAnalyticsResponse` contains:

Table 35: Analytics response structure

Field group	Content
Totals	totalSubmissions, acceptedSubmissions, acceptanceRate, uniqueProblemsSolved
Rating	CodePulse Rating, tier, per-platform ratings
Streaks	currentStreak, longestStreak, activeDays
Topic data	topicBreakdown (accepted per topic), weaknessScores (0.0–1.0 per topic)
Difficulty	difficultyBreakdown: Beginner / Easy / Medium / Hard / Expert
Activity	activityHeatmap: map of ISO date → submission count (last 365 days)
Verdicts	verdictDistribution: ACCEPTED, WRONG_ANSWER, TLE, ...
Platform data	platformBreakdown: per-platform solved count, acceptance rate, verdicts, topic coverage
Trend	ratingTrend: list of {date, rating} from Codeforces

8.6 Recommendation Implementation

`RecommendationServiceImpl.generate()` runs the selection algorithm of Section 6.8.4 inside a transaction: it deletes the user's previous recommendations, selects up to ten unsolved problems and persists them with a human-readable `reason` (for example "Targets your weak topic: Dynamic Programming") and a priority `score`. `markSolved` and `dismiss` simply flip boolean flags, so an item disappears from the active list but remains for audit and future feedback weighting.

8.7 Frontend Implementation

- **Routing:** Next.js App Router pages for each route in Section 7.2; an authenticated layout renders the sidebar and redirects to login on a 401.
- **State:** a Zustand store holds the JWT and basic user info; TanStack Query caches server data (profile, analytics, recommendations, AI session) and is invalidated after sync, solve or dismiss actions.
- **API client:** a thin wrapper adds the `Authorization` header and normalises errors to toast notifications.
- **Charts:** Recharts components for platform comparison, rating trend, difficulty, weakness radar, verdict donut; a custom grid for the activity heatmap.
- **Hidden insights:** a pure function that maps the analytics response to an ordered list of finding objects (category, icon, title, text).
- **Motion:** Framer Motion for staggered insight cards and `AnimatePresence` exits when items are solved or dismissed.

8.8 Implementation Challenges and Resolutions

Table 36: Implementation problems and fixes

Problem	Symptom	Resolution
Spring AI SDK incompatibility	<code>UnrecognizedPropertyException</code> on every AI call after the provider added <code>prompt_tokens_details</code>	Replaced the SDK with <code>DirectMistralService</code> using <code>RestTemplate</code> and <code>JsonNode</code>

Problem	Symptom	Resolution
<code>LazyInitializationException</code>	HTTP 500 on analytics, recent submissions and recommendations	<code>JOIN FETCH</code> in repository queries plus <code>@Transactional(readOnly = true)</code> on service / controller methods
LLM output with Markdown fences or truncation	JSON parse failures	Strict system prompt (JSON only, never truncate), fence stripping, and <code>buildError()</code> fallback
Platform rate limits	Occasional empty sync from LeetCode	Per-platform error isolation; retry on next scheduled run or manual sync
Stale handles on dashboard	Handles missing after profile save	Profile and dashboard always fetch the profile from the server and invalidate the cache after save (FR-4)
Unbounded data growth	Slower analytics over time	<code>SubmissionPruningService</code> with 300-row ceiling

8.9 Source Control and Coding Practices

- Source is hosted on GitHub: <https://github.com/Badhon1401/SPL-3>.
- Layered package structure, constructor injection and interface-based services.
- DTOs separate API contracts from entities; entities are never serialised directly.
- Consistent error contract through `ApiResponse` and `GlobalExceptionHandler`.
- Configuration externalised; Lombok reduces boilerplate.

9. AI Engineering Design

9.1 Goals

The AI Coach lets a user describe a practice goal in plain language — "I keep failing DP problems; give me a progression from easy to hard" — and receive problems chosen **for that person**, not generic lists. The LLM is therefore never asked a context-free question: it always receives the user's measured performance profile.

9.2 Pipeline



Figure 27: AI pipeline flow

Table 37: AI pipeline stages

Stage	Action	Output
1. Context assembly	Collect rating, per-platform stats, top 8 weak topics with failure rates, top 5 strong topics, difficulty distribution, last 15 accepted titles, last 5 WA/TLE struggle topics	Markdown context string
2. Prompt engineering	Prepend context to the user prompt and a strict JSON schema; system prompt enforces JSON-only output	System + user messages
3. LLM call	POST to <code>api.mistral.ai/v1/chat/completions</code> through <code>RestTemplate</code> (OpenRouter as fallback)	Raw JSON response
4. Parsing and validation	Walk the <code>JsonNode</code> tree for recommendations, coach insight and focus areas; catch malformed output	Validated domain objects or error
5. Persistence	Deactivate previous session; save new session and items (<code>saveAll</code>) in one transaction	Session DTO returned

9.3 Context Payload

An illustrative (abbreviated) context block sent with the prompt:

```
## Performance profile
CodePulse Rating: 1624 (Specialist)   Problems solved: 187   Acceptance: 54%
Streak: current 9 / longest 21
## Platform stats
Codeforces: 92 solved, est. rating 1580 | LeetCode: 71 solved | CodeChef: 24 solved
## Weak topics (failure rate)
1. Dynamic Programming 62%   2. Graphs 48%   3. Binary Search 41%   ...
## Strong topics
Math, Implementation, Greedy ...
## Recently solved (last 15) and recent WA/TLE topics ...
## User request
"I keep failing DP problems - give me a progression from easy to hard." (count = 6)
```

9.4 Model Selection

Table 38: Model selection

Model	Role	Reason
<code>mistral-large-latest</code>	Primary	Strong knowledge of competitive-programming problems across the four platforms; reliable JSON-only output when instructed; competitive pricing with a free tier at console.mistral.ai
OpenRouter (any model)	Fallback	Switch to DeepSeek, Gemini or any OpenAI-compatible model with one property change; same <code>RestTemplate</code> + <code>JsonNode</code> code path

9.5 Prompt Design

The system prompt enforces four absolute rules: (1) return **only valid, complete JSON**; (2) never truncate; (3) recommend only **real problems with real URLs**; (4) personalise the `reason` field by referencing the user's specific weakness or goal. The user message appends a schema template that specifies exactly the fields and number of items expected.

```
{
  "coachInsight": "string - 2-4 sentences on the user's current state",
  "focusAreas": ["string", "..."],
  "recommendations": [
    {
      "title": "string", "platform": "CF|LC|AC|CC", "url": "https://...",
      "difficulty": "string", "estimatedRating": 0,
      "topics": ["string"], "reason": "string", "timeEstimate": "string"
    }
  ]
}
```

9.6 Error Handling and Safety

- **Schema drift:** `JsonNode` parsing ignores unknown fields, so provider-side additions do not break the pipeline (NFR-7).
- **Malformed output:** any parse exception is caught and logged; `buildError()` returns an empty session with an explanatory coach insight; the **previous session is preserved**.
- **Provider failure:** network errors, invalid keys and rate limits follow the same graceful path; OpenRouter can be enabled as fallback.
- **Hallucinated problems:** the prompt demands real URLs and the UI always links to the platform page so the user can verify the problem exists; unverified items can be dismissed.
- **Privacy:** only aggregate performance data and recent problem titles are sent. Passwords, tokens and email addresses are never included.

9.7 Limitations

- LLM knowledge of very recent problems may be incomplete; the algorithmic engine provides a database-backed alternative.
- Answer quality depends on prompt clarity and on the 300-submission window of data.
- Latency depends on the external provider; the UI shows an animated generating state.

10. Testing

10.1 Testing Approach and Strategy

10.1.1 Objectives

- Validate all 22 functional requirements through unit, integration, API and acceptance testing.
- Verify the CodePulse Rating formula for edge cases (no platform, one platform, all platforms).
- Confirm that the AI pipeline handles provider errors gracefully.
- Verify deduplication, pruning, JWT protection and role-based access.

10.1.2 Levels of Testing

Table 39: *Testing levels*

Level	Scope	Tools
Unit	Rating calculator, streak logic, weakness scoring, pruning logic, JSON parsing	JUnit 5, Mockito
Integration	Repository queries (<code>JOIN FETCH</code>), service + database interaction, security filter chain	Spring Boot Test, TestContainers (PostgreSQL), <code>spring-security-test</code>
API	All 17 endpoints — happy paths and error cases	Postman collection
System / acceptance	All pages end to end in Chrome and Firefox	Manual acceptance checklist
Regression	Re-run affected endpoint after each fix (LazyInit, Mistral JSON)	Postman, JUnit

10.1.3 Test Environment

Table 40: *Test environment*

Item	Configuration
Backend	Java 17, Spring Boot 3.3.4, Maven 3.9, port 8080
Database	PostgreSQL 15 (local instance and TestContainers for automated tests)
Frontend	Node.js 18+, Next.js 14, port 3000
Browsers	Google Chrome and Mozilla Firefox (latest)
External	Live Codeforces / LeetCode APIs for sync tests; Mistral API key for AI tests; pre-seeded data for rate-limited scenarios

10.2 Item Pass / Fail Criteria

Table 41: *Pass / fail criteria*

Verdict	Criteria
PASS	The API returns the expected HTTP status and JSON structure; database state matches expectation; UI behaves as specified; no exception stack trace is returned to the client.
FAIL	Any HTTP 500; any <code>LazyInitializationException</code> in logs; incorrect data; unhandled Mistral error; duplicate or un-pruned submissions; protected endpoint reachable without a valid token.
Suspension	Testing of a feature is suspended if the database is unreachable or the build fails; resumes after the blocker is fixed.

10.3 Risks and Contingencies

Table 42: Risks and contingencies

Risk	Likelihood	Impact	Contingency
LeetCode GraphQL rate-limits	Medium	Medium	Retry later with back-off; test with pre-seeded database data
AtCoder kenkoooo API unavailable	Low	Low	Catch and log; other platforms continue; pre-seeded AtCoder submissions
Mistral API key invalid or quota exhausted	Low	Medium	<code>buildError()</code> graceful response; previous session kept; OpenRouter fallback
<code>LazyInitializationException</code> regression	Low (fixed)	High	<code>JOIN FETCH</code> in repositories and <code>@Transactional</code> ; integration tests guard it
Test database contamination	Medium	Medium	<code>@Transactional</code> rollback per test or <code>@DirtiesContext</code> ; <code>TestContainers</code> per class
Platform API schema change	Low	Medium	Tolerant parsing; per-platform isolation; fix in one service class
Demo-day network failure	Medium	High	Pre-synchronised demo account and recorded AI session available offline

10.4 Test Cases and Results

Twenty-five test cases cover authentication, synchronisation, analytics, recommendations, the AI Coach, access control and usability.

Table 43: Test cases

TC	Description	Input / steps	Expected result	Result
01	Register with valid data	POST <code>/api/auth/register</code> with unique email and username	201 + JWT token	PASS
02	Register duplicate email	Same email twice	400 "Email already registered"	PASS
03	Login wrong password	Correct email, wrong password	401 "Invalid credentials"	PASS
04	Access analytics without token	GET <code>/api/analytics/me</code> , no Authorization header	401 Unauthorized	PASS
05	Save four platform handles	PUT <code>/api/users/me</code> with all handles	200; handles visible in GET <code>/me</code>	PASS
06	Handles pre-filled on profile page	Open <code>/profile</code> after TC-05	All four inputs show saved handles	PASS
07	Handles visible on dashboard	Open <code>/dashboard</code> after TC-05	Platform grid shows handles + external links	PASS
08	Codeforces sync deduplication	Sync twice with same submissions	Count unchanged after second sync	PASS
09	Pruning at 300 ceiling	Insert 310 submissions for one user	Count = 300; oldest 10 removed	PASS
10	Analytics without LazyInit error	GET <code>/api/analytics/me</code> after sync	200 + full JSON; no 500 in logs	PASS
11	Recent submissions without LazyInit	GET <code>/api/submissions/recent?limit=12</code>	200 + list; no exception	PASS
12	Analytics with zero platforms	No handles; GET <code>/api/analytics/me</code>	200; all zeros; empty platform breakdown	PASS
13	CodePulse Rating bounds	User with one LeetCode easy problem solved	Rating in [800, 1250]	PASS

TC	Description	Input / steps	Expected result	Result
14	Curated recommendation generation	POST <code>/api/recommendations/generate</code>	200; ≤ 10 unsolved problems	PASS
15	Recommendations without LazyInit	GET <code>/api/recommendations</code>	200 + list; no exception	PASS
16	Mark recommendation solved	PATCH <code>/api/recommendations/{id}/solved</code>	200; item absent from next GET	PASS
17	AI prompt submission	POST <code>/api/ai/recommend</code> with valid prompt	200; session with ≥ 1 item	PASS
18	AI session persistence	GET <code>/api/ai/sessions/latest</code> after TC-17	Same session without regeneration	PASS
19	AI regeneration deactivates old	Submit a second prompt for the same user	Only the new session is active	PASS
20	Mistral error handling	Invalid API key configured	200 with empty items + error coach insight	PASS
21	Streak across two platforms	3 CF submissions day 1, 2 LC submissions day 2	currentStreak = 2	PASS
22	Topic weakness ranking	10 DP attempts / 8 fail; 5 graph / 2 fail	DP weakness > graph weakness	PASS
23	Admin lists users	GET <code>/api/admin/users</code> with ADMIN token	200 + user list	PASS
24	Admin endpoint with USER role	GET <code>/api/admin/users</code> with USER token	403 Forbidden	PASS
25	Problem links clickable	Click a problem title on dashboard / recommendations	Opens correct platform URL in new tab	PASS

10.5 Detailed Test Outcomes (Selected Cases)

TC-01 / TC-02 / TC-03 — Registration and Login

Table 44: Detail: authentication tests

Field	Authentication tests
Preconditions	Database empty of the test user; backend running.
Steps	1. POST register with <code>{username, email, password, fullName}</code>. 2. POST register again with the same email. 3. POST login with a wrong password.
Expected outcome	1. HTTP 201; body contains <code>token, type = Bearer, userId, username, email, role</code>; <code>users</code> row has a BCrypt hash (starts with <code>\$2</code>). 2. HTTP 400 with message "Email already registered"; no new row. 3. HTTP 401 "Invalid credentials"; no token.
Result	All three behaved as expected — PASS.

TC-08 / TC-09 — Deduplication and Pruning

Table 45: Detail: synchronisation tests

Field	Synchronisation tests
Preconditions	User with a valid Codeforces handle; empty <code>submissions</code> table for that user.
Steps	1. Trigger sync; note the count N. 2. Trigger sync again. 3. For TC-09, insert 310 synthetic submissions and run the pruning service.

Field	Synchronisation tests
Expected outcome	1. N rows stored (≤ 200 fetched). 2. Count remains N (duplicates skipped by <code>platformSubmissionId</code>). 3. Count is exactly 300 and the 10 rows with the earliest <code>submitted_at</code> are gone.
Result	Counts matched; the surviving rows were the 300 most recent — PASS.

TC-10 / TC-11 / TC-15 — Lazy-Loading Regression

Table 46: Detail: lazy-loading regression

Field	Lazy-loading regression tests
Preconditions	User with synchronised submissions; OSIV disabled.
Steps	Call analytics, recent-submissions and recommendations endpoints; inspect application log.
Expected outcome	HTTP 200 with complete JSON including problem titles and topics; no <code>LazyInitializationException</code> ; a single joined SQL statement for submission loading.
Result	No exception observed after the <code>JOIN FETCH</code> fix — PASS.

TC-17 to TC-20 — AI Coach

Table 47: Detail: AI Coach tests

Field	AI Coach tests
Preconditions	Valid Mistral key (TC-17–19); invalid key (TC-20); user has synchronised data.
Steps	1. POST prompt "Give me 6 DP problems from easy to hard". 2. GET latest session. 3. POST a second prompt. 4. Replace key with an invalid value and POST again.
Expected outcome	1. 200; session with ≥ 1 item, coach insight and focus areas. 2. Identical session returned without calling the provider. 3. Exactly one <code>active = true</code> session in the table. 4. 200 with empty items and an explanatory coach insight; previous session retained; no stack trace.
Result	Outcomes as expected — PASS.

TC-23 / TC-24 — Access Control

Table 48: Detail: access-control tests

Field	Role-based access tests
Steps	Call <code>GET /api/admin/users</code> with an ADMIN token, then with a USER token.
Expected outcome	ADMIN → 200 with list of users; USER → 403 Forbidden.
Result	PASS.

10.6 Requirements Traceability

Table 49: Requirements-to-test traceability

Requirement	Test cases	Requirement	Test cases
FR-1, FR-2	TC-01, 02, 03	FR-13	TC-10, 12
FR-3, FR-4	TC-05, 06, 07	FR-14, FR-15	TC-14, 15
FR-5, FR-6	TC-08	FR-16	TC-16
FR-7	TC-09	FR-17, FR-18, FR-19	TC-17, 20
FR-8	Shares sync path with TC-08, 09	FR-20	TC-18

Requirement	Test cases	Requirement	Test cases
FR-9	TC-12, 13	FR-21	TC-19
FR-10	TC-21	FR-22	Manual acceptance (AI page)
FR-11	TC-22	NFR-2	TC-04, 23, 24
FR-12	Manual acceptance (Analytics page)	NFR-3	TC-20 and per-platform error isolation

10.7 Test Summary

Table 50: *Test summary*

Metric	Value
Test cases defined	25
Passed	25
Failed	0
Defects found and fixed during testing	LazyInitializationException on three endpoints; Mistral JSON parsing failure after provider schema change
Areas needing continued attention	Automated tests for frontend components; load testing beyond 300 submissions per user

11. User Manual

This manual explains how an end user installs nothing but a browser: once the system is running (Chapter 12), everything is done through the web interface at `http://localhost:3000`.

11.1 Getting Started

11.1.1 Create an Account

1. Open `http://localhost:3000`. You are redirected to the **Login** page.
2. Click **Create Account**. Step 1 asks for a username and your full name; click **Next**.
3. Step 2 asks for your email and a password; click **Launch**.
4. You are signed in automatically and land on the Dashboard.

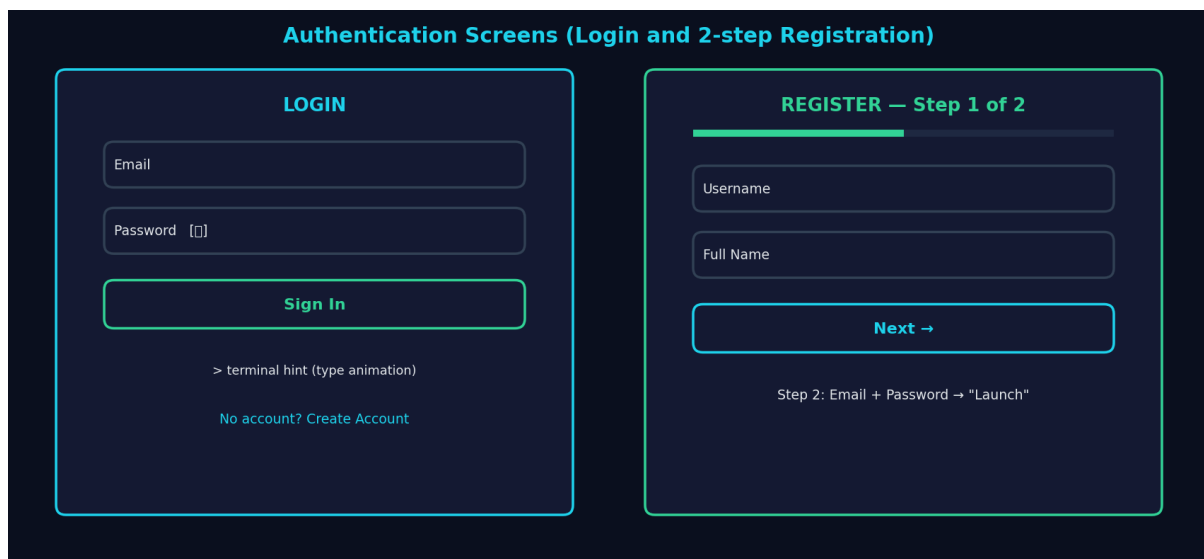


Figure 28: Login and registration screens

11.1.2 Sign In

Enter your email and password and press **Sign In**. A wrong password shows an "Invalid credentials" message. Your session lasts 24 hours; after that you are sent back to the login page.

11.1.3 Connect Your Platforms

1. Click **Profile** in the sidebar.
2. Enter your handle for each platform you use — for example `tourist` for Codeforces. You may connect one, several or all four.
3. Click **Save Changes**. Connected platforms show a green status dot, and a link to your public profile.

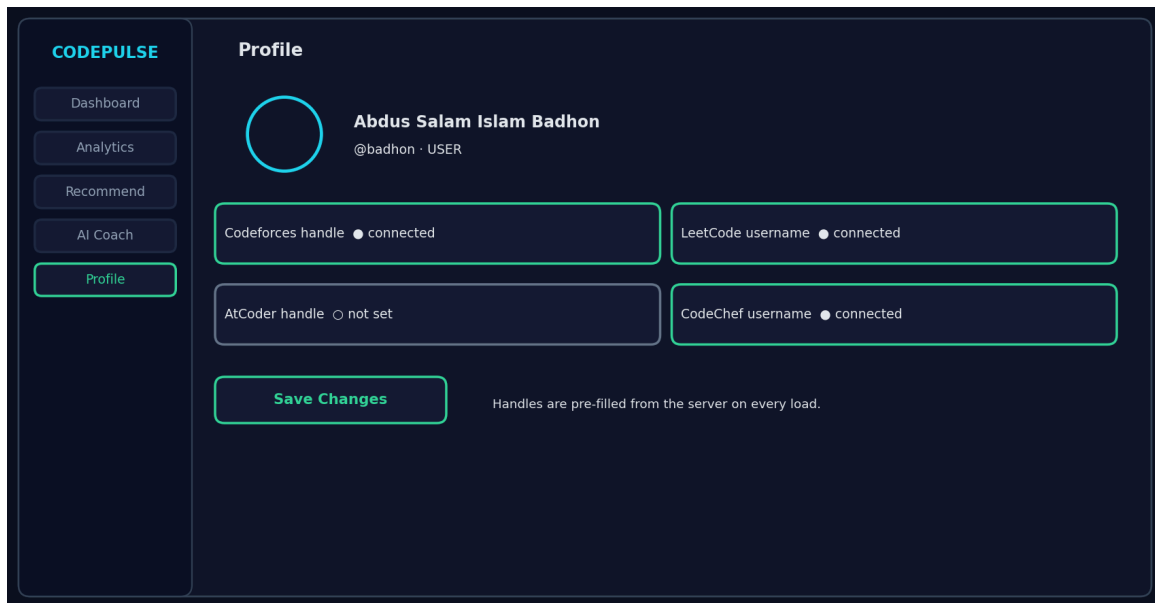


Figure 29: Profile screen

11.1.4 Synchronise Your Data

1. Open the **Dashboard**.
2. Click **Sync All Platforms**. A spinner shows while data is fetched in the background.
3. After a few seconds the page refreshes with your submissions, rating and streak.

CodePulse also synchronises automatically every 6 hours. Each sync fetches up to 200 recent submissions per platform and the system keeps your latest 300 submissions in total.

11.2 Using the Application

11.2.1 Dashboard

The Dashboard is your overview. The four tiles show your **CodePulse Rating**, **problems solved**, **current streak** and **acceptance rate**. The Platform Connection Grid shows each connected handle with its solved count; click a handle to open your profile on that site, or "Add handle →" to connect a new one. The Recent Submissions list shows your latest attempts with verdict badges; click a title to open the problem.



Figure 30: Dashboard

11.2.2 Analytics

The Analytics page explains *why* your numbers look the way they do.

- **Insights panel:** automatically generated findings, colour-coded as Critical, Warning, Success, Tip or Pattern — for example "You have 0 accepted DP submissions — Blind Spot".
- **Charts:** platform comparison, Codeforces rating trend, difficulty distribution, weakness radar, verdict distribution and a 365-day activity heatmap (hover a cell for date and count).
- **Topic coverage:** all topics you have solved problems in, sorted by volume.

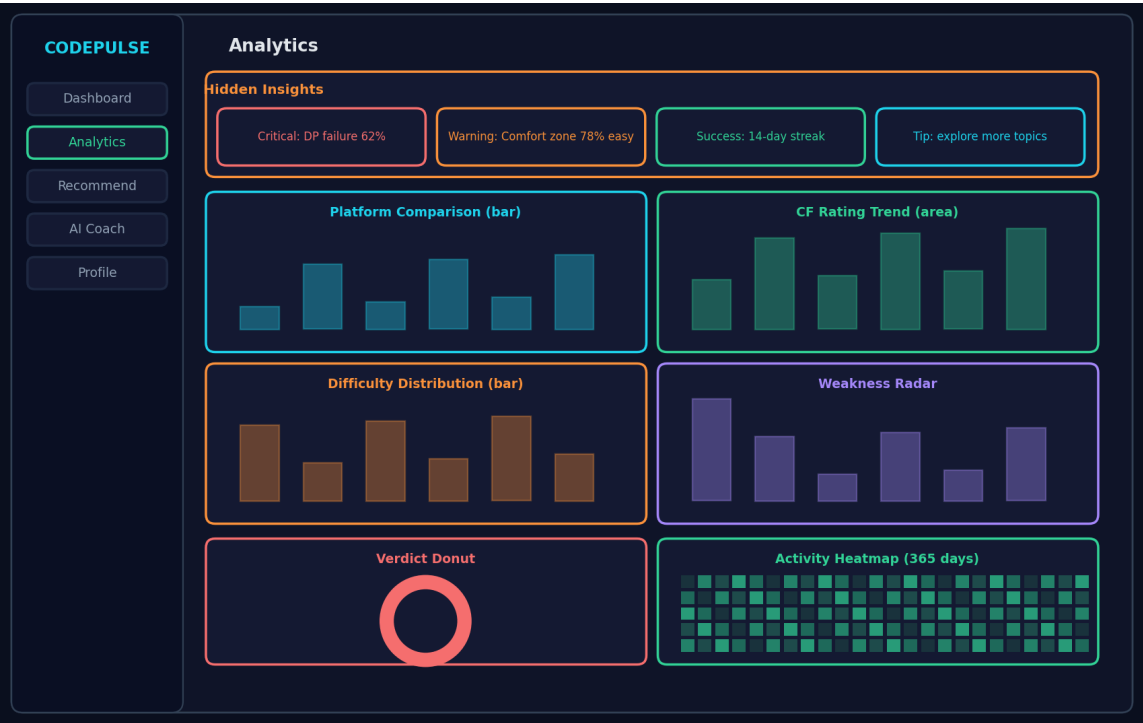


Figure 31: Analytics

11.2.3 Curated Recommendations

1. Open **Recommendations** and click **Generate New**. Problems are chosen from your weakest topics at a suitable difficulty; older recommendations are replaced.
2. Use the **All / CF / LC / AC / CC** tabs to filter by platform.
3. Click a problem title to open it on the platform.
4. Click **✓ Solved** once you have solved it, or **✗ Dismiss** to hide it. This feedback refines future suggestions.



Figure 32: Recommendations

11.2.4 AI Coach

1. Open **AI Coach** and describe your goal, e.g. "I keep failing DP problems — give me a progression from easy to hard." You can also click an example chip.
2. Choose how many problems you want (4, 6 or 8) and click **Ask AI** (or press Ctrl+Enter).
3. Wait while the animated dots show that the coach is working.
4. Read the **Coach Insight**, the **Focus Areas** and the recommended problems, each with a reason written for you.
5. Solve or dismiss items. Your latest session is shown whenever you return; ask again with a new prompt to replace it.

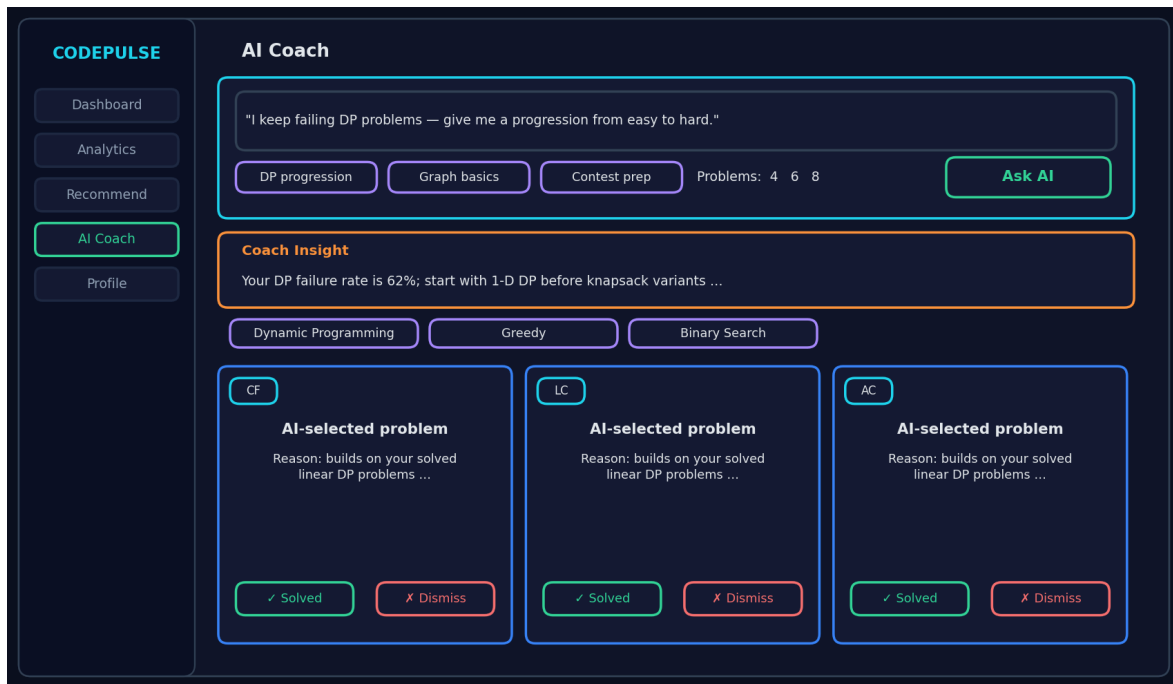


Figure 33: AI Coach

11.2.5 Profile

Edit your full name, avatar URL and any of the four handles, then click **Save Changes**. Changes appear immediately on the Dashboard because every page loads your profile fresh from the server.

11.2.6 Administrator Functions

Users with the ADMIN role can list all users and deactivate accounts through the admin API (Chapter 13). Ordinary users receive 403 Forbidden for these endpoints.

11.3 Understanding the Numbers

Table 51: Glossary of displayed metrics

Term	Meaning
CodePulse Rating	A single Elo-like score (800–3800) that blends your estimated rating on every connected platform, weighted by how much you practise on each.
Acceptance rate	Accepted submissions divided by total submissions in your stored window.
Current / longest streak	Consecutive days with at least one submission on any platform.
Weakness score	Failure rate on a topic: 0.0 means strong, 1.0 means you fail every attempt.
Difficulty tiers	Beginner, Easy, Medium, Hard and Expert, mapped from each platform's own scale.

11.4 Troubleshooting

Table 52: Troubleshooting guide

Issue	Resolution
Handles not showing on the Dashboard	Click Save Changes on the Profile page, then return to the Dashboard.
Sync shows 0 new submissions	Check the handle spelling on the Profile page. LeetCode occasionally rate-limits; wait a minute and retry.
AI recommendations are empty	Check the Mistral API key in <code>application.properties</code> . If the key is valid, try the OpenRouter fallback by adding <code>app.openrouter.api-key</code> .

Issue	Resolution
"Invalid credentials" on login	Check email and password; passwords are case-sensitive.
Redirected to login unexpectedly	Your 24-hour session expired; sign in again.
500 Internal Server Error	Check the backend log; ensure you are running the latest build with the lazy-loading fixes.
Charts are empty	You have no synchronised data yet — connect a handle and click Sync All Platforms.

12. Installation and Deployment Guide

No separate installer is provided. The application is installed from source or by running the packaged executable JAR together with the Next.js application.

12.1 Prerequisites

Table 53: Prerequisites

Software	Version	Purpose
JDK	17 or newer	Run the Spring Boot backend
Maven	3.9+	Build the backend
PostgreSQL	15+	Database
Node.js	18+	Build and run the frontend
Git	Any recent	Clone the repository
Mistral API key	—	AI Coach (free tier at console.mistral.ai)

12.2 Obtain the Source

```
git clone https://github.com/Badhon1401/SPL-3.git
cd SPL-3
```

12.3 Backend

1. Create the database:

```
CREATE DATABASE codepulse;
```

2. Generate a JWT secret (Base64, 256-bit):

```
openssl rand -base64 32
```

3. Edit `src/main/resources/application.properties`:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/codepulse
spring.datasource.username=postgres
spring.datasource.password=YOUR_PASSWORD
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
jwt.secret=YOUR_BASE64_SECRET
jwt.expiration=86400000
spring.ai.mistralai.api-key=YOUR_MISTRAL_KEY
```

4. Run:

```
mvn spring-boot:run
```

The server starts on <http://localhost:8080>. The 20 algorithm topics are seeded automatically on first boot. To produce a deployable artifact:

```
mvn clean package -DskipTests
java -jar target/codepulse-1.0.0.jar
```

12.4 Frontend

```
cd frontend
npm install
cp .env.local.example .env.local      # set
NEXT_PUBLIC_API_URL=http://localhost:8080
npm run dev
```

Open `http://localhost:3000`; you are redirected to the login page. For a production build use `npm run build` followed by `npm start`.

12.5 Verifying the Installation

Table 54: Smoke test

Check	Expected result
Backend log	Application started on port 8080; topics seeded
<code>POST /api/auth/register</code> via Postman	201 with token
Open <code>http://localhost:3000</code>	Login page renders
Register, save a handle, click Sync	Dashboard shows submissions and rating

12.6 Production Considerations

- Supply secrets (`jwt.secret`, DB password, API keys) through environment variables or a secret manager.
- Serve both tiers behind HTTPS (for example an Nginx reverse proxy) and restrict CORS to the real frontend origin.
- Set `spring.jpa.hibernate.ddl-auto=validate` and manage schema changes with a migration tool (e.g. Flyway) once the schema is stable.
- Back up the PostgreSQL database regularly.

13. API Reference

All endpoints except `/api/auth/**` require the header `Authorization: Bearer <token>`. Errors use a common `ApiResponse` envelope with an HTTP status code and message.

13.1 Endpoint Summary

Table 55: REST endpoints

Method	Endpoint	Description	Auth
POST	<code>/api/auth/register</code>	Create an account	Public
POST	<code>/api/auth/login</code>	Authenticate, receive JWT	Public
GET	<code>/api/users/me</code>	Get own profile with handles	JWT
PUT	<code>/api/users/me</code>	Update profile and handles	JWT
GET	<code>/api/analytics/me</code>	Full combined analytics	JWT
POST	<code>/api/analytics/sync</code>	Trigger multi-platform sync (async)	JWT
GET	<code>/api/submissions/recent</code>	Recent submissions, all platforms (<code>?limit=</code>)	JWT
GET	<code>/api/recommendations</code>	Active curated recommendations	JWT
POST	<code>/api/recommendations/generate</code>	Generate new recommendations	JWT
PATCH	<code>/api/recommendations/{id}/solved</code>	Mark solved	JWT
PATCH	<code>/api/recommendations/{id}/dismiss</code>	Dismiss	JWT
POST	<code>/api/ai/recommend</code>	AI Coach: generate from prompt	JWT
GET	<code>/api/ai/sessions/latest</code>	Load stored AI session	JWT
PATCH	<code>/api/ai/items/{id}/solved</code>	Mark AI item solved	JWT
PATCH	<code>/api/ai/items/{id}/dismiss</code>	Dismiss AI item	JWT
GET	<code>/api/admin/users</code>	List all users	JWT + ADMIN
DELETE	<code>/api/admin/users/{id}</code>	Deactivate user	JWT + ADMIN

13.2 Request and Response Examples

13.2.1 Register / Login

```
POST /api/auth/register
{ "username": "badhon", "email": "badhon@example.com",
  "password": "S3cret!pass", "fullName": "Abdus Salam Islam Badhon" }

201 Created
{ "token": "eyJhbGciOi...", "type": "Bearer", "userId": 1,
  "username": "badhon", "email": "badhon@example.com", "role": "ROLE_USER" }
```

13.2.2 Update Profile

```
PUT /api/users/me
{ "fullName": "Abdus Salam", "codeforcesHandle": "tourist",
  "leetcodeHandle": "myhandle", "atcoderHandle": "me_ac",
  "codechefHandle": "me_cc", "avatarUrl": "https://..." }

200 OK -> updated profile (all fields optional in the request)
```

13.2.3 Analytics (abbreviated)

```
GET /api/analytics/me
200 OK
```

```
{ "totalSubmissions": 240, "acceptedSubmissions": 130, "acceptanceRate": 54.2,
  "uniqueProblemsSolved": 112, "codePulseRating": 1624,
  "currentStreak": 9, "longestStreak": 21,
  "topicBreakdown": { "dp": 14, "graphs": 9 },
  "weaknessScores": { "dp": 0.62, "graphs": 0.48 },
  "difficultyBreakdown": { "Beginner": 20, "Easy": 40, "Medium": 40, "Hard": 10,
    "Expert": 2 },
  "activityHeatmap": { "2026-09-30": 5 },
  "verdictDistribution": { "ACCEPTED": 130, "WRONG_ANSWER": 70 },
  "ratingTrend": [ { "date": "2026-08-01", "rating": 1580 } ] }
```

13.2.4 AI Coach

```
POST /api/ai/recommend
{ "prompt": "I keep failing DP problems - progress easy to hard", "count": 6 }

200 OK
{ "sessionId": 7, "coachInsight": "...", "focusAreas": ["Dynamic Programming"],
  "items": [ { "id": 31, "title": "...", "platform": "CF", "url": "https://...",
    "difficulty": "1400", "topics": ["dp"], "reason": "..." } ] }
```

The JSON bodies above are illustrative; field names follow the README and DTO naming conventions. Verify exact names against the DTO classes before quoting them in the defence.

13.3 Status Codes

Table 56: HTTP status codes

Code	Meaning	Typical cause
200 / 201	Success / created	Normal operation, registration
400	Bad request	Validation failure; duplicate email
401	Unauthorized	Missing, invalid or expired JWT; wrong credentials
403	Forbidden	USER role calling an admin endpoint
404	Not found	Unknown recommendation, item or user id
500	Server error	Unexpected failure (treated as a test failure)

14. Project Management

14.1 Development Phases

Table 57: Development phases

Phase	Activities	Duration	Status
1	Requirements, SRS, database schema, entity modelling	Weeks 1–2	Complete
2	Auth module, JWT security, User entity, profile API	Weeks 3–4	Complete
3	Codeforces and LeetCode sync; Problem / Topic / Submission entities	Weeks 5–6	Complete
4	AtCoder and CodeChef sync; combined analytics; CodePulse Rating	Weeks 7–8	Complete
5	Recommendation engine, pruning service, scheduled sync	Week 9	Complete
6	AI Coach: direct Mistral HTTP, context builder, session persistence	Weeks 10–11	Complete
7	Next.js frontend: Dashboard, Analytics, Recommendations, AI Coach, Profile	Weeks 12–14	Complete
8	Bug fixes (LazyInit, Mistral JSON), testing, hidden insights engine	Weeks 15–16	Complete
9	Final report, code review, demonstration preparation	Week 17	In progress

14.2 Gantt Chart

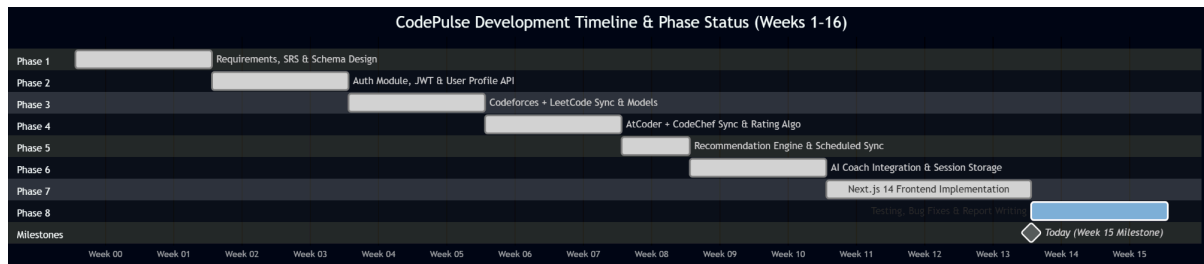


Figure 34: Gantt chart of project phases

14.3 Risk Management

Table 58: Project risks

Risk	Mitigation actually applied
Third-party API instability	Per-platform service isolation; error logging; retry on next schedule
LLM format drift	Direct HTTP + <code>JsonNode</code> ; strict JSON prompt; graceful fallback
Schedule pressure (single developer)	Backend-first approach; frontend after stable APIs; fixed weekly milestones
Scope creep	Out-of-scope list in Section 1.5; contest scheduling and social features deferred

14.4 Lessons Learned

- Explicit fetch strategies (`JOIN FETCH`) are preferable to relying on open-session-in-view.
- External SDKs at milestone versions are a risk; a thin HTTP client plus tolerant parsing is more resilient.
- Treat each external integration as an unreliable dependency: isolate, time-out, log and continue.
- Documenting design decisions early (SRS, diagrams) made the final report largely a consolidation exercise.

15. Conclusion and Future Work

15.1 Summary of Achievements

CodePulse meets its objectives: it aggregates four competitive-programming platforms into one analytical view, computes a unified rating, identifies topic-level weaknesses, generates insights that users would not otherwise notice, recommends problems algorithmically and — through a grounded LLM pipeline — converts free-text goals into personalised practice plans. The system is secured with JWT and BCrypt, protects its database from unbounded growth, tolerates partial failure of external services and is covered by a 25-case test suite in which all cases pass.

15.2 Limitations

- Analytics reflect the latest 300 submissions rather than complete history.
- Community-maintained AtCoder and CodeChef APIs may change or become unavailable.
- AI recommendations can occasionally reference problems that do not exist; users must verify through the provided link.
- No automated end-to-end tests for the frontend yet.

15.3 Future Work

- Add more platforms (HackerRank, Kattis, CSES) using the one-service-per-platform pattern.
- Weight recommendation scoring with long-term feedback from solved / dismissed history.
- Contest calendar and reminders; goal tracking and weekly digests by email.
- Caching and background pre-computation of analytics for very large histories.
- Containerisation with Docker Compose and a CI pipeline (build, test, deploy).
- Optional social features: friend comparison and leaderboards.

15.4 Repository and Deliverables

Table 59: *Final deliverables*

Deliverable	Location
Public version-controlled repository	https://github.com/Badhon1401/SPL-3
Backend source and README	Repository root (Maven project)
Frontend source	<code>frontend/</code> directory
Final report	This document
Installer	Not applicable — run from source / executable JAR (Chapter 12)
Demonstration (acceptance test)	Live demo using the scenarios in Chapter 10 and Chapter 11

References

1. Codeforces API documentation — <https://codeforces.com/apiHelp>
2. LeetCode GraphQL endpoint — <https://leetcode.com/graphql>
3. kenkoooo AtCoder Problems API — <https://kenkoooo.com/atcoder/>
4. Mistral AI API documentation — <https://docs.mistral.ai>
5. OpenRouter documentation — <https://openrouter.ai/docs>
6. Spring Boot Reference Documentation, version 3.3 — <https://docs.spring.io/spring-boot/>
7. Spring Security Reference — <https://docs.spring.io/spring-security/reference/>
8. Next.js Documentation — <https://nextjs.org/docs>
9. PostgreSQL 15 Documentation — <https://www.postgresql.org/docs/15/>
10. Roger S. Pressman and Bruce R. Maxim, **Software Engineering: A Practitioner's Approach**, McGraw-Hill.
11. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
12. RFC 7519 — JSON Web Token (JWT).

Appendix A: Glossary

Table 60: *Glossary*

Term	Definition
AC / WA / TLE	Accepted / Wrong Answer / Time Limit Exceeded — submission verdicts
CF / LC / AC / CC	Codeforces / LeetCode / AtCoder / CodeChef
DTO	Data Transfer Object — API request / response class
JWT	JSON Web Token used for stateless authentication

Term	Definition
LLM	Large Language Model (Mistral AI)
OSIV	Open Session In View — JPA session kept open during view rendering
ORM / JPA	Object-Relational Mapping / Java Persistence API
Pruning	Deleting the oldest submissions beyond the per-user ceiling
Weakness score	Failure rate per topic (0.0–1.0)

Appendix B: Environment Variables and Properties Checklist

Table 61: *Configuration checklist*

Item	Required	Notes
Database URL, user, password	Yes	PostgreSQL 15 database <code>codepulse</code>
<code>jwt.secret</code>	Yes	Base64 256-bit; never commit
<code>jwt.expiration</code>	No	Default 86400000 ms
Mistral API key	For AI Coach	Free tier available
OpenRouter API key	Optional	Fallback model
<code>codepulse.submissions.max-per-user</code>	No	Default 300
Frontend <code>NEXT_PUBLIC_API_URL</code>	Yes	Backend base URL